

《人工智能导论》期末开卷考试速查手册

公式默认包含偏置时会明确写出；卷积尺寸均先取整（通常向下取整）
ver. 261110-GPT5.5xHRaw+F3E5 by. MucheXD+LaobieXL

0. 30 秒定位表

题干关键词	直接翻到
三大学派、伦理、智能体、Python 库	§1
RDF、三元组、图神经网络、产生式、归结	§2
BFS/DFS/UCS、A*、8 数码、Minimax、 α - β 、MCTS	§3
TSP、染色体、适应度、轮盘赌、PMX	§4
粒子、速度、惯性、个体/社会认知	§5
学习类型、损失、数据划分、混淆矩阵、ROC/AUC、过拟合	§6
线性回归、Ridge/Lasso、逻辑回归、极大似然、Softmax	§7
KNN、最大间隔、支持向量、核函数、软间隔	§8
K-means、K-means++、轮廓系数、PCA	§9
熵、条件熵、信息增益、ID3/C4.5/CART、随机森林	§10
MDP、Q-learning、 ϵ -greedy、折扣因子	§11
神经元、激活函数、三种学习规则、前馈/反馈	§12
BP 参数/连接/FLOPs、链式法则、梯度消失/爆炸	§13
CNN 输出尺寸/参数/连接/FLOPs、VGG/Inception/ResNet	§14
GAN、模式崩溃	§15
RNN、BPTT、LSTM、GRU	§16
Attention、Q/K/V、PE、LN、残差、FFN、Transformer	§17
LLM 训练、幻觉、Prompt/微调/RAG、Agent	§18
贝叶斯网络、关联规则、模拟退火、TF-IDF、模糊推理	§19
高频计算模板与答题句式	§20

1. 人工智能基本概念、三大学派、伦理、智能体与常用工具

1.1 基本概念

- **人工智能 (AI)**：研究、开发用于模拟、延伸和扩展人的智能的理论、方法、技术与系统；研究内容可概括为机器感知、机器思维、机器学习、机器行为。
- **人类智能特征**：感知能力、记忆与思维能力、学习能力、行为能力；知识是一切智能行为的基础，智力体现获取并运用知识解决问题的能力。
- **图灵测试**：从外显行为判断机器能否表现出与人难以区分的智能；它是行为标准，不等同于证明机器拥有意识。
- **弱人工智能/窄人工智能**：面向特定任务；**AGI**：希望在广泛任务上具备通用迁移与推理能力。当前课程中的算法均属于专用智能技术。
- **常见应用**：计算机视觉、自然语言处理、语音识别、推荐、机器人、智能决策、知识图谱。
- **计算智能**强调启发式、近似与自适应求解，常包括进化计算、人工神经网络、模糊计算和群体智能。

1.2 三大学派

学派	核心观点/表示	典型方法	优势与局限
符号主义（逻辑主义）	智能来自符号、知识与逻辑推理；显式规则	谓词逻辑、产生式系统、专家系统、知识图谱	可解释、适合规则明确；知识获取困难、处理噪声与感知较弱
连接主义	智能来自大量简单单元的连接及权重学习	人工神经网络、深度学习、BP、CNN、Transformer	表征学习能力强；依赖数据与算力、解释和可靠性较难
行为主义（进化/控制学派）	智能由“感知—行动—反馈”及适应环境产生	强化学习、遗传算法、群体智能、机器人控制	强调交互、自主与适应；试错成本高、训练稳定性受环境影响

1.3 智能体 Agent

- **定义**：通过传感器感知环境、通过执行器采取行动的实体；**理性智能体**在给定感知序列和已有知识下，选择使期望性能度量最大的动作。
- **环境维度**：完全/部分可观察，确定/随机，情节式/序贯，静态/动态，离散/连续，单智能体/多智能体。
- **结构类型**：简单反射型（只看当前感知）→基于模型型（维护内部状态）→目标型（依据目标规划）→效用型（比较结果效用）→学习型（从经验改进）。简单反射型无法保证在所有环境中最优。
- **特性**：自主性、反应性、主动性、社会性。

1.4 人工智能伦理

风险	核心问题	常见治理措施
公平与偏见	训练数据或设计造成群体差异	代表性数据、分组评估、公平约束、持续审计
隐私与安全	个人数据泄露、模型攻击与滥用	最小化收集、脱敏/权限控制、安全测试、用途限制
透明与可解释	决策理由难以追溯	模型卡、数据说明、解释工具、保留人工复核
责任与可控	事故责任、自动化越权	明确责任链、人工在环、可停止/回滚、日志留痕
可靠与诚实	幻觉、漂移、错误置信	外部验证、置信提示、监控、红队测试、限制高风险自动决策
就业与社会影响	岗位替代、数字鸿沟、权力集中	再培训、影响评估、普惠设计、合规治理

1.5 常见 Python/AI 库（本手册不重复语法）

任务	库/框架
数值与表格	NumPy、SciPy、Pandas
可视化	Matplotlib（及 Seaborn，课程若提及）
传统机器学习	Scikit-learn
深度学习	PyTorch、TensorFlow/Keras；课件还列 Caffe、MXNet、PaddlePaddle、MindSpore 等
NLP/LLM	Transformers、Tokenizers 等
图与知识图谱	NetworkX、Neo4j/Cypher、RDFLib/Jena/SPARQL
部署/加速	ONNX、TensorRT、OpenVINO、NCNN

2. 知识表示与知识图谱

2.1 数据—信息—知识与知识表示

- **数据**是符号记录；**信息**是赋予语境和意义的数据；**知识**可概括为事实、规则与概念；**智能**体现获取、组织、应用知识以完成目标。
- 知识具有**相对正确性与不确定性**；不确定性可来自随机性、模糊性、经验性、信息不完全。
- 好的知识表示应：①表示能力充分；②支持推理和计算；③便于组织、维护和扩展；④便于人理解和机器实现。

2.2 经典知识表示（低篇幅覆盖）

方法	形式与特点	常考点
命题逻辑	命题只有真/假，用 $\neg$ 、 $\wedge$ 、 $\vee$ 、 $\rightarrow$ 连接	表达结构能力有限，不能自然表达对象、关系与量词
一阶谓词逻辑	谓词、个体常量、变量、函数、 $\forall$ 、 $\exists$	自然、精确、严密；可表示对象属性和关系，难直接表示不确定知识
产生式	IF 条件 THEN 结论/动作 【置信度】	系统由规则库、推理机、综合数据库/工作记忆组成；循环“匹配→冲突消解→执行”
框架	对典型对象建立框架，槽 slot 存属性，侧面 facet 存约束/默认值	适合结构化、继承性知识；槽值可以指向另一个框架
语义网络/知识图谱	节点表示实体/概念，边表示关系	结构清晰、便于关系查询和推理
专家系统	用专家知识与推理解决领域复杂问题	人机接口、知识库、推理机、数据库、知识获取、解释机构；瓶颈是知识获取

有效推理：只要前提都真，结论必真。 $P \rightarrow Q$, $P \Rightarrow Q$ （肯定前件）有效； $P \rightarrow Q$, $\neg Q \Rightarrow \neg P$ （否定后件）有效；由 Q 推 P 的“肯定后件”无效。

归结证明步骤：①把待证结论取否；②与前提合取；③化为子句集/CNF；④寻找互补文字并归结；⑤推出空子句 $\square$ ，说明原前提蕴含结论。未推出空子句不能直接断言结论为假，可能是搜索或规则应用未完成。

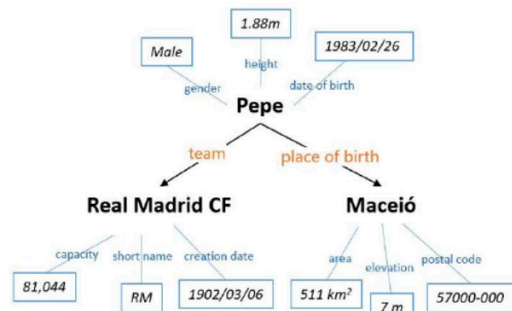
2.3 知识图谱核心

- Google 于 2012 年推广“Things, not strings”思路：由字符串匹配转向实体及其关系理解。
- **知识图谱**本质上是**有向语义图**：节点为实体 entity 或概念 concept，边为实体/概念之间的语义关系；也可把属性和值表示为边和对象。
- **基本单元**：关系事实（头实体，关系，尾实体）；属性事实（实体，属性，属性值）。
- **概念区分**：实体是具体对象；概念/类别组织实体；关系连接实体；属性描述实体；公理/模式约束类型、关系及规则。
- **类型**：语言知识图谱、常识知识图谱、世界知识图谱、认知知识图谱、领域知识图谱。

2.4 RDF 图【大题操作】

- **RDF 三元组**：<Subject, Predicate, Object>；Subject 是主语资源，Predicate 是关系/属性，Object 可为实体资源或字面量。
- **画图规则**：实体/概念/字面对象画为节点；谓语或关系写在有向边上；箭头由主语指向宾语；一句话可拆成多条三元组；代词先消解为明确实体。
- **执行步骤**：①圈出实体、概念和属性值；②找每个陈述中的谓语/关系；③将复合句拆成原子事实；④统一实体名称并消解“他/它/其”；⑤逐条写三元组；⑥按“主语→关系→宾语”画有向边；⑦检查方向、重复与遗漏。
- **例**：“图灵是数学家，提出图灵测试，国籍是英国；英国别名大不列颠。”可写：（图灵,isA,数学家）、（图灵,提出,图灵测试）、（图灵,国籍,英国）、（英国,别名,大不列颠）。

- 类似于万维网发布的**RDF (Resource Description Framework)**，其基本构成单元是事实，每个事实被表示为一个形如 **<Subject, Predicate, Object>** 即 **<主语, 谓语, 宾语>**的三元组。
- **主语**: 通常是实体、事件或者概念中的任何一个
- **谓语**: 通常是关系或者属性
- **宾语**: 实体、事件、概念，或者普通的值（如数字、字符串等）



RDF图是由三元组组成的有向图

RDF Data		
<S,	P,	O>
<Pepe,	gender,	male>
<Pepe,	height,	1.88m>
<Pepe,	date of birth,	1983/02/06>
<Pepe,	team,	Ream Madrid CF>
<Pepe,	place of birth,	Maceió>
<Maceió,	area,	511km²>
<Maceió,	elevation,	7m>
<Maceió,	postal code,	57000-000>
...		
...		

三元组表

2.5 RDF 与属性图、构建与使用

项	RDF 图	属性图 Property Graph
基本形式	所有事实均为三元组，W3C 标准	节点和边均可直接携带属性
查询	SPARQL	Cypher、Gremlin
侧重	语义互操作、本体与逻辑推理	高效遍历和工程图应用
常用系统	Jena、RDF 存储	Neo4j 等图数据库

- **知识图谱构建流程**: 多源数据获取 → 知识抽取（实体、关系、属性）→ 实体对齐/消歧 → 知识融合 → 质量评估 → 图存储 → 持续更新。
- **来源**: 结构化表格、半结构化网页/JSON/XML、非结构化文本/图像；文本抽取常用于补全图谱。
- **两层结构**: 模式层/本体层定义概念、关系、约束；数据层保存具体事实。
- **存储**: 关系表适合规整数据；图数据库采用邻接关系，适合多跳查询；工程中常混合使用。
- **推理/补全**: 逻辑规则与 ILP、路径方法、知识表示学习（TransE、RESCAL 等）、神经网络、混合方法。
- **GNN 与知识图谱**: GNN 通过邻居消息传递聚合关系信息；图结构可动态插入实体、关系和新知识。知识图谱是数据/知识表示，GNN 是在图上学习的模型，二者层次不同。

3. 搜索算法、A*、博弈搜索与蒙特卡洛树搜索

3.1 搜索问题与盲目搜索

- 搜索问题通常由**初始状态**、**动作/后继函数**、**目标测试**、**路径代价**构成；状态空间图可能有环，图搜索需维护已访问/ CLOSED 集。
- 记分支因子为 b 、最浅目标深度为 d 、最大深度为 m 。

算法	选择规则/数据结构	完备性	最优性	主要特点
BFS 广度/宽度优先	先进先出队列，逐层扩展	有限 b, d 时完备	单步代价相同时最优	时间、空间约 $O(b^d)$ ，内存大
DFS 深度优先	栈/递归，先走到底	无限深或有环时不保证	不保证	空间约 $O(bm)$ ，内存小；需深度限制/判重

算法	选择规则/数据结构	完备性	最优性	主要特点
UCS 一致代价	每次扩展累计代价 g 最小节点（优先队列）	步代价正且有下界时完备	是	边权不等时相当于 Dijkstra；填空常考
深度受限	DFS 加最大深度 l	目标深度 $\leq l$ 时	不保证	避免无限下潜
迭代加深	依次做深度 $0,1,2,\dots$ 的 DFS	有限 b,d 时	单步代价相同时	BFS 的完备性 + DFS 的低空间

3.2 启发式搜索与 A* 【计算】

- **启发函数** $h(n)$ ：估计从当前节点到目标的剩余最小代价；利用问题知识指导扩展。
- **贪婪最佳优先** 只看 $h(n)$ ，通常快但不保证最优；**A*** 用 $f(n)=g(n)+h(n)$ ，其中 $g(n)$ 是起点到 n 的实际历史代价， $h(n)$ 是未来估计。
- $h(n)=0$ 时 A* 退化为 UCS/Dijkstra；若所有边代价相同，扩展次序可与 BFS 一致。
- **可采纳 admissible**： $0 \leq h(n) \leq h^*(n)$ ，不高估真实最优剩余代价；树搜索下可保证 A* 最优。
- **一致 consistent**： $h(n) \leq c(n, n') + h(n')$ ，并令目标 $h=0$ ；图搜索时更强，保证沿路径 f 不下降，节点首次从 OPEN 取出即可确定最优 g 。
- 在均可采纳前提下， h 越接近 h^* ，通常扩展节点越少；高估可能更快，但会失去最优保证。

A* 落笔步骤：①建立 $OPEN=\{\text{起点}\}$ 、 $CLOSED=\emptyset$ ，记 $g(\text{start})=0, f=g+h$ ；②从 OPEN 取 f 最小节点；③若为目标，沿父指针回溯；④移入 CLOSED；⑤生成后继，算新 $g'=g(n)+c(n,m)$ ；⑥未见过则记录 $g,h,f,parent$ 并入 OPEN；已在 OPEN 且 g' 更小则更新；必要时重新打开 CLOSED 中被改进节点；⑦重复至找到目标或 OPEN 为空。遇同 f 按题目规定顺序处理。

3.3 8 数码常见启发函数

- **有解性**：把空格去掉，将棋盘按行展开，计算逆序对数；对 3×3 八数码，初态与目标的逆序对奇偶性相同才可达。
- h_1 ：位置错误的数字块个数（通常不计空格）； h_2 ：各数字块到目标位置的曼哈顿距离之和。
- h_1, h_2 均不高估时可采纳；通常 $h_2 \geq h_1$ 且更接近真实代价，A* 扩展更少。

3.4 极小极大 Minimax

- 适用典型假设：双人、轮流、零和、确定、完全信息、双方理性。
- 流程（课件原文）：1) 生成博弈树（深度限制）2) 评估叶子节点 3) 自底向上计算节点估值 4) 选择最优动作
- 终局效用从叶向上回传：MAX 节点取子节点最大值，MIN 节点取最小值。深度过大时在截止层使用评价函数 $Eval(s)$ 。
- 递归式： $V(s)=\max_a V(\text{Result}(s,a))$ (MAX)， $V(s)=\min_a V(\text{Result}(s,a))$ (MIN)。根节点选择使最坏情况下收益最大的动作。

3.5 Alpha-Beta (α - β) 剪枝 【画图大题】

- α ：当前路径上 MAX 已保证的最好值，即下界，初始 $-\infty$ ； β ：MIN 已保证的最好值，即上界，初始 $+\infty$ 。
- MAX 节点更新 $\alpha=\max(\alpha,v)$ ；MIN 节点更新 $\beta=\min(\beta,v)$ ；一旦 $\alpha \geq \beta$ ，剩余兄弟分支不可能改变祖先决策，立即剪去。
- 等价表述：在 MIN 节点若 $\beta \leq \text{祖先}\alpha$ ，剪余下子树；在 MAX 节点若 $\alpha \geq \text{祖先}\beta$ ，剪余下子树。
- **不改变 Minimax 结果**；只减少计算。结点排序越接近“MAX 先看大值、MIN 先看小值”，剪枝越多；最优排序复杂度近 $O(b^{(d/2)})$ ，最坏仍 $O(b^d)$ 。

纸面操作：①根标 $\alpha=-\infty, \beta=+\infty$ ；②按指定方向深搜到叶；③回传一个叶值便更新所在 MAX/MIN；④每更新一次检查 $\alpha \geq \beta$ ；⑤在被剪边上画叉，写出触发时的 α, β ；⑥最终按 Minimax 回传根值。

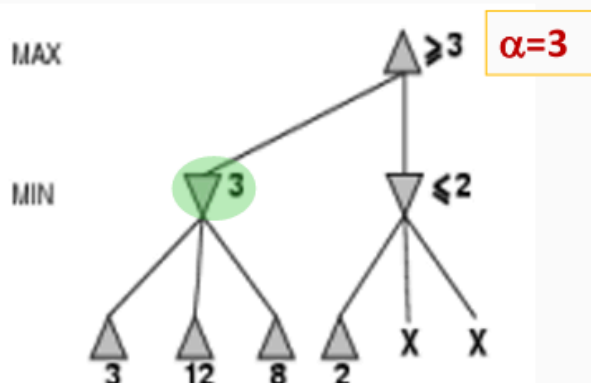
模拟卷型：若先算 $B1=\min(3,12,8)=3$ ，根 $\alpha=3$ ；进入 $B2$ 首叶为 2，使 $\beta=2 \leq \alpha=3$ ，可剪 $B2$ 后续叶 4、6。

■ MAX节点的评估下限值 α

作为正方出现的MAX节点，假设它的MIN子节点有N个，那么当它的第一个MIN子节点的评估值为 α 时，则对于其它的子节点，如果有高过 α 的，就取那最高的值作为该MAX节点的评估值；如果没有，则该MAX节点的评估值为 α 。

总之，该MAX节点的评估值不会低于 α ，这个 α 就称为该MAX节点的评估下限值。

- ◆极大 (MAX) 节点的下界 (MAX 玩家至少能得到的分数) 为 α
- ◆极小 (MIN) 节点的上界 (MIN 玩家至多能得到的分数) 为 β



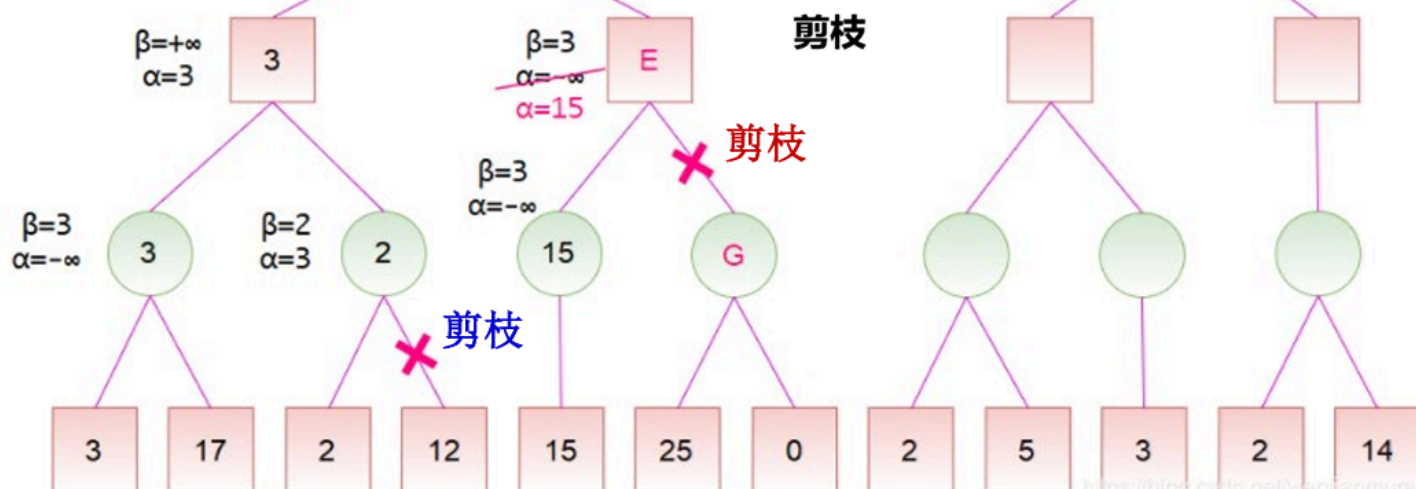
▲ α 剪枝样例，利用已经确定下来的下界 α 值，去截断 (prune) 一个 MIN 节点的后续搜索

更新 α 值 红框: MAX, 取大

更新 β 值

绿圆: MIN, 取小

深度限制的深度优先搜索
(假设深度为4、同层从左往右搜索) + 估价函数 + 剪枝



- Min 节点: 其 β 值 $\leq$ 祖先节点的 α 值时，剪掉该节点的后续子节点， α 剪枝
- Max 节点: 其 α 值 $\geq$ 祖先节点的 β 值时，剪掉该节点的后续子节点， β 剪枝

3.6 蒙特卡洛树搜索 MCTS

阶段	动作
Selection 选择	从根按 UCT/UCB 反复选择子节点，直到到达未完全扩展节点
Expansion 扩展	加入一个或多个尚未访问的合法动作节点
Simulation/Rollout 模拟	从新节点按随机或轻量策略走到终局/截断，得到回报
Backpropagation 反传	沿访问路径更新访问次数 N 、累计收益 w 、均值 $Q=w/N$

- 常用 $UCT_i = W_i/N_i + c\sqrt{(\ln N_p/N_i)}$ ：第一项利用高平均收益，第二项探索访问较少节点； c 越大越鼓励探索。
- 优点：可随时停止并输出当前最好动作，适合分支巨大、难设计精确评价函数的博弈；模拟越多通常越稳定。有限模拟下不保证找到全局最优。
- 与 A：A 解决路径代价问题并依赖启发函数；MCTS 通过采样估计动作价值，常用于博弈/决策树。

4. 基本遗传算法、旅行商问题 TSP、编码、适应度与 PMX

4.1 基本概念与流程

- 遗传算法（GA）是受自然选择和遗传机制启发的群体随机优化方法；同时维护多个候选解，适合非线性、不可微、离散或组合优化。
- 对应关系：**基因**=解的一个变量/片段；**染色体/个体**=完整候选解；**种群**=候选解集合；**适应度**=个体优劣；**代**=一次群体更新。
- 流程：编码 → 随机/启发式初始化种群 → 计算适应度 → 选择 → 交叉 → 变异 → 形成新种群（可精英保留） → 判断终止条件。
- 终止：达到最大代数、达到目标适应度、若干代无改进、时间/预算耗尽。

GA的特点：

- 只对参数集的**编码**进行操作，而非对参数本身进行操作；
- 从许多初始点开始**并行搜索**，搜索的全局性强；
- 通过**目标函数**计算适应度，对问题的依赖性小；
- 寻优规则是由**概率**决定的，而非确定性的；
- 在解空间进行高效**启发式**搜索，非盲目地穷举或完全随机搜索；
- 具有**并行计算**的特点，适合于复杂问题的优化；
- **计算简单，功能强大**。

4.2 编码

编码	适用	要点
二进制编码（位串编码）	连续变量离散化、0/1 选择	完整、合法、尽量无冗余；可能有汉明悬崖
实数编码	连续优化	直接表示参数，避免量化误差，需实数交叉/变异
整数/排列编码（有序串编码）	TSP、排序、调度	每个城市/任务恰出现一次，普通交叉易重复或缺失

二进制解码：长度 n 位串 A 对区间 $[x_{\min}, x_{\max}]$ ， $x = x_{\min} + \text{dec}(A)(x_{\max} - x_{\min}) / (2^n - 1)$ 。精度要求决定 n 。

TSP 染色体：长度为城市数的排列；每个城市出现且仅出现一次；闭环距离需加“末城市→首城市”。

4.3 适应度函数

- 原则：与优化目标单调一致、可区分个体、数值稳定、尽量非负、计算代价可接受，并对非法解施加惩罚或修复。
- 最大化且 $f \geq 0$ 可直接 $\text{Fit} = f$ ；最小化可用 $\text{Fit} = 1/(1+f)$ ($f \geq 0$)、 $1/f$ ($f > 0$)、 $C-f$ 或排名适应度。
- TSP：先算总路程 L ，常取 $\text{Fit} = 1/(1+L)$ ；路径越短，适应度越高。

- **欺骗问题**：适应度景观将搜索引向错误区域；**早熟收敛**：个体过早相似，陷入局部最优（原文：进化初期，个别超级个体控制选择过程）；**停滞**：进化末期，个体差异太小导致陷入局部极值。

目标函数 $f(x)$	适应度函数 $Fit(x)$
$\max f(x), \quad f(x) \geq 0$	$Fit(x) = f(x)$ 例：最大化投资收益
$\min f(x), \quad f(x) > 0$	$Fit(x) = \frac{1}{f(x)}$ 例：最小化行驶路径
$\max f(x)$	$Fit(x) = \begin{cases} f(x) - C_{\min} & f(x) > C_{\min} \\ 0 & \text{其他情况} \end{cases}$
$\min f(x)$	$Fit(x) = \begin{cases} C_{\max} - f(x) & f(x) < C_{\max} \\ 0 & \text{其他情况} \end{cases}$

4.4 选择、交叉、变异

算子	作用	常见方法/注意
选择	让高适应度个体更有机会繁殖	轮盘赌 $p_i = Fit_i / \sum Fit$ 、锦标赛、排序选择；选择压力太大易早熟
交叉	重组父代已有优良片段	单点、多点、均匀、算术；排列用 PMX/OX 等合法交叉
变异	随机引入新基因、维持多样性	位翻转、实数扰动、排列交换/逆序；概率通常小
精英保留	防止当前最好解被随机算子破坏	直接复制少量最优个体，一般不宜过多

其他选择口径：(μ, λ) 从 μ 个父代产生的 λ 个后代中只选 μ 个后代； $\mu + \lambda$ 从父代与后代合集中选 μ 个，后者保留历史好解更稳。排序选择按名次预设概率，减轻超级个体引起的过早收敛；适应度尺度变换也用于控制选择压力。

课程常用经验范围：种群规模约 20–200； $p_c \approx 0.85-1$ ； $p_m \approx 0.001-0.15$ ，具体依编码与问题调节。 p_m 太小易失多样性，太大近似随机搜索。

4.5 PMX 部分匹配交叉【重点大题】

目的：排列编码普通片段交换会造成重复/缺失；PMX 保留交换片段并通过映射修复区间外冲突。

标准步骤：①选两个切点 $[l, r]$ ；②把父代 2 的中段复制到子代 1、父代 1 的中段复制到子代 2；③按同一位置建立双向映射对；④对子代区间外原来自对应父代的基因逐一检查：若与中段冲突，沿映射链反复替换，直到得到未出现在中段的基因；⑤填入并检查每个城市恰一次。

模拟卷例： $P1 = [1, 2, 3, 4, 5]$ ， $P2 = [2, 5, 4, 1, 3]$ ，区间 2–4；中段 $[2, 3, 4]$ 与 $[5, 4, 1]$ ，映射 $2 \leftrightarrow 5$ 、 $3 \leftrightarrow 4$ 、 $4 \leftrightarrow 1$ 。

- 子代 1 中段取 P2： $[_, 5, 4, 1, _]$ 。P1 区间外首位 1 冲突： $1 \rightarrow 4 \rightarrow 3$ ，填 3；末位 5 冲突： $5 \rightarrow 2$ ，填 2；得 $[3, 5, 4, 1, 2]$ 。
- 子代 2 中段取 P1： $[_, 2, 3, 4, _]$ 。P2 首位 2 冲突： $2 \rightarrow 5$ ，填 5；末位 3 冲突： $3 \rightarrow 4 \rightarrow 1$ ，填 1；得 $[5, 2, 3, 4, 1]$ 。

$$P_1 = (3 \ 4 \ 8 \mid 2 \ 7 \ 1 \mid 6 \ 5),$$

1. 从父代随机选择两个个体 P_1, P_2 和两个点 $P_2 = (4 \ 2 \ 5 \mid 1 \ 6 \ 8 \mid 3 \ 7)$.

$$O_1 = (\times \ \times \ \times \mid 1 \ 6 \ 8 \mid \times \ \times),$$

2. 将 P_1, P_2 两点之间部分提取出来, 放在子代 O_2 , O_1 相同位置 $O_2 = (\times \ \times \ \times \mid 2 \ 7 \ 1 \mid \times \ \times)$.

3. 在提取出来的部分中, 映射关系为 $2 \rightarrow 1, 7 \rightarrow 6, 1 \rightarrow 8$ 。下面我们可以把子代未填充的部分补充完整 (使用父代), O_1 第一个位置来源于 P_1 第一个位置, 是3, 第二个位置是4, 最后一个位置5, 因为没有与已经填上的部分 (1 6 8) 重复, 直接填上就可以了, 同理 O_2

$$O_1 = (3 \ 4 \ \times \mid 1 \ 6 \ 8 \mid \times \ 5),$$

也可以填上。 $O_2 = (4 \ \times \ 5 \mid 2 \ 7 \ 1 \mid 3 \ \times)$.

此处的替换根据映射链依次向后, 映射链是 P_1-P_2 交叉区建立的联系
本题中, 8 的映射链是 8-1-2, 顺着映射链可以找到未被占用的符号

4. O_2 , O_1 中打叉的部分在父代与子代相同位置重复, 因此要使用映射的方法填入。例如, O_1 第三个位置应填入8, 而8应该在第六个位置了, 因此用1替换, 而1也被填入了, 则用2替换, 所以最后填入 O_1 第三个位置的是2, 同理, 第7个位置应填入7.

$$O_1 = (3 \ 4 \ 2 \mid 1 \ 6 \ 8 \mid 7 \ 5), \quad O_2 = (4 \ 8 \ 5 \mid 2 \ 7 \ 1 \mid 3 \ 6).$$

4.6 GA 常见改进与答题

- 早熟改进: 提高/自适应变异、适度降低选择压力、随机注入新个体、保持多样性/拥挤与小生境、多种群并行、精英+扰动、混合局部搜索。
- 优点: 无需梯度、全局并行搜索、适配复杂编码; 缺点: 收敛可能慢、参数敏感、适应度计算昂贵、有限代不保证全局最优。
- OX 顺序交叉: 保留一段父代片段, 按另一父代的相对顺序填剩余位置; PMX 更偏向保留城市的绝对位置, OX 更偏向保留相对顺序。教师提示 OX 考查概率低, 优先掌握 PMX。

改进算法:

- 双倍体 GA 用显性/隐性染色体提供了一种记忆以前有用的基因块的功能, 双倍体遗传算法采用显性遗传, 按照显性染色体的选择概率进行选择操作, 双倍体遗传延长了有用基因块的寿命, 提高了算法的收敛能力, 在变异概率低的情况下能保持一定水平的多样性。
- 建立两个遗传算法群体, 分别独立地运行复制、交叉、变异操作, 同时当每一代运行结束以后, 选择两个种群中的随机个体及最优个体分别交换 (打破原种群内的平衡态达到更高的平衡态, 有利于算法跳出局部最优), 在交叉算子、变异算子基础上加一个杂交算子 (迁移操作);
- 自适应 GA 让较优个体使用较小 p_c, p_m 、较差个体使用较大概率; 多目标 NSGA-II 以非支配排序和拥挤度保持 Pareto 解多样性。对于适应度高于群体平均适应度 (f_{avg}) 的个体, 选择较低的, 使得 P_c, P_m 该解得以保护进入下一代; 对低于平均适应度 (f_{avg}) 的个体, 选择较高的 P_c, P_m , 使该解被淘汰, 进而产生新个体。

GA 的一般特征: 对编码后的解操作、以目标函数评价、使用概率转移、群体并行搜索; 无法保证有限代得到全局最优。

5. 粒子群优化 PSO: 概念、公式与参数

5.1 动机与概念

- PSO 源于鸟群/鱼群协同觅食; 每个粒子表示一个候选解, 通过速度连续移动, 保留方向与历史记忆。相较 GA 的世代式选择—交叉—变异, PSO 更新更直接, 连续优化中常收敛较快。
- x_i : 粒子当前位置; v_i : 速度; $pbest_i$: 粒子自身历史最好位置; $gbest$: 全群历史最好位置; 适应度用于比较位置优劣。

5.2 核心公式【必背】

对粒子 i 的第 j 维:

速度更新公式:

$$v_{ij}(t+1) = wv_{ij}(t) + c_1r_1[p_{ij} - x_{ij}(t)] + c_2r_2[p_{gj} - x_{ij}(t)]$$

速度=惯性+个体历史最优经验+群体历史最优经验

位置更新公式:

$x_{ij}(t+1) = x_{ij}(t) + v_{ij}(t+1)$
其中 $r_1, r_2 \sim U(0, 1)$ ；第一项为惯性项，第二项为个体认知项，第三项为社会认知项。

参数	作用与趋势
w 惯性权重	大：保持速度、全局探索强；小：局部开发强。常随迭代线性递减
c1 个体学习因子	向自身经验靠近；过大时粒子较独立、振荡可能增强
c2 社会学习因子	向群体最优靠近；过大易快速聚集、早熟
r1, r2	随机数，每次迭代随机生成，为了随机化轨迹维持探索
Vmax	限制单步速度，常取搜索范围的约 10%–20%；过大易越界，过小移动慢
粒子数	常见约 20–40；太少覆盖不足，太多计算开销大

特殊情形：c1=0 只有社会学习，易快速聚集；c2=0 只有个体学习，缺少群体共享；二者均 0 只按惯性飞行。
c2>c1 更强调群体引导，课件指出可用于多峰问题，但过强仍可能丢失多样性；常见初值 $c_1 \approx c_2 \approx 2$ ，需结合惯性/收缩因子保证稳定。

5.3 算法步骤

①随机初始化位置与速度并做边界约束；②计算适应度；③更新每粒子的 pbest；④更新 gbest；⑤按公式更新 v, x ；⑥速度/位置越界则截断、反弹或重采样；⑦满足最大迭代、目标精度或长期无改进时停止。

对比	GA	PSO
信息更新	选择、交叉、变异产生后代	原速度、自身最优、群体最优合成新速度
记忆	通常通过种群间接保留	显式保存 pbest/gbest
典型变量	离散、组合、连续均可	原生适合连续，二进制 PSO 可扩展离散
风险	早熟、参数多、收敛慢	早熟、群体多样性丢失、参数/拓扑敏感

二进制 PSO 常用 $\text{sigmoid}(v)$ 将速度转成取 1 概率；速度通常需限幅（如 $[-4, 4]$ ）。蚁群算法教师明确提示不作为本次重点。

6. 机器学习基本概念、评估、数据不平衡、过拟合与欠拟合

6.1 学习范式与任务

范式	数据/反馈	目标与例子	典型算法
监督学习	有标签 (x, y)	分类、回归	线性/逻辑回归、KNN、SVM、树、神经网络
无监督学习	无标签 x	发现结构：聚类、降维、关联	K-means、PCA、关联规则
半监督学习	少量标签+大量无标签	降低标注成本	伪标签、自训练、一致性学习
强化学习	与环境交互，只有奖励	序贯决策、最大长期回报	Q-learning、DQN、策略梯度

- **回归**输出连续值（房价、温度）；**分类**输出离散类别或概率（肿瘤良恶性）；**聚类**无标签分组；**降维**把高维表示映射到低维。
- 机器学习三要素：**模型**（函数集合）、**策略/准则**（损失与目标）、**算法**（优化求解）。

6.2 标准建模流程

问题与指标定义 → 数据收集 → 缺失/异常/噪声处理 → 特征提取/构造/选择 → 编码、标准化/归一化 → 划分训练/验证/测试 → 训练与超参数搜索 → 验证选择 → 测试集一次性评估 → 部署监控。

- **标准化** $z=(x-\mu)/\sigma$ ；**Min-Max 归一化** $(x-\min)/(\max-\min)$ ；KNN、K-means、SVM、PCA、梯度优化对尺度敏感，树模型相对不敏感。
- 类别型特征：无序类别常用 One-Hot；缺失值可按均值/中位数/众数、模型插补或单独缺失类别处理。
- **数据泄漏**：测试/验证信息进入训练，如先用全数据标准化、用测试集调参；会导致虚高评估。所有预处理参数应仅由训练集拟合。

6.3 损失、风险与优化

名称	含义
Loss	单个样本预测误差，如平方、绝对值、0-1、对数损失
Cost/经验风险	训练集平均损失
结构风险/正则项	控制复杂度，如 L1/L2
Objective	经验风险 + 正则项等完整优化目标

- 回归常用平方损失/MSE、MAE；分类训练常用交叉熵；0-1 损失直观但不可导。
- 梯度下降： $\theta \leftarrow \theta - \alpha \nabla J(\theta)$ ；学习率过大可能振荡/发散，过小收敛慢。BGD 用全数据稳定但慢；SGD 单样本噪声大；Mini-batch 最常见。
- **参数**由训练学习（权重、偏置）；**超参数**由人/搜索确定（学习率、树深、K、正则强度）。

6.4 数据集划分与评估

- **留出法**：互斥划分训练/验证/测试，分类应尽量分层保持类别比例；测试集只用于最终评估。
- **K 折交叉验证**：分成 K 份，轮流 1 份验证、K-1 份训练，取 K 次均值；小数据更稳定但训练 K 次。分类用 Stratified K-fold。
- **自助法 Bootstrap**：从 m 个样本有放回抽取 m 次构造训练集；单个样本未被抽中的概率约 $(1-1/m)^m \approx e^{-1} \approx 0.368$ ，约 36.8% 可作包外样本。
- 超参数选择常用网格搜索/随机搜索 + 交叉验证；最终测试集不可参与调参。

6.5 混淆矩阵与指标【必算】

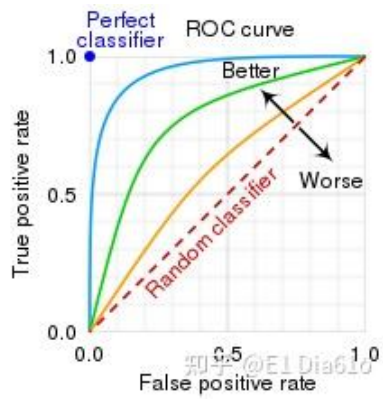
以“正类”为关注对象：真正正且预测正 TP；真正正预测负 FN；真实负预测正 FP；真实负预测负 TN。

指标	公式	侧重点/场景
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$	类别较均衡、误判代价接近
Precision/查准率	$TP/(TP+FP)$	预测为正时有多可信；假阳性代价高，如垃圾邮件误杀
Recall/TPR/查全率	$TP/(TP+FN)$	真正例找回多少；漏诊代价高，如疾病筛查
Specificity/TNR	$TN/(TN+FP)$	负类识别能力
FPR	$FP/(FP+TN)=1-TNR$	正常样本被误报比例
F1	$2PR/(P+R)=2TP/(2TP+FP+FN)$	不平衡/长尾且需平衡 P、R

- **数据不平衡**：多数类可使 Accuracy 很高但少数类几乎识别不到；优先看 P/R/F1、PR 曲线、ROC-AUC、混淆矩阵，必要时用类别权重、欠采样/过采样/SMOTE、阈值调整、分层采样。
- **Macro**：先对每类分别算指标再等权平均，重视少数类；**Micro**：先汇总所有类 TP/FP/FN 再算，受多数类影响大；Weighted 按类样本数加权。

6.6 ROC 与 AUC

- 改变分类阈值得到不同 (FPR, TPR)，横轴 FPR、纵轴 TPR；曲线越靠**左上角**越好；对角线近似随机猜测。
- **AUC** 为 ROC 下方面积，可解释为随机抽一个正样本和负样本时，模型把正样本评分排在负样本前的概率；1 最好、0.5 近随机。极端不平衡时 PR 曲线通常更敏感。



6.7 回归指标

- $MSE=(1/m)\sum(\hat{y}-y)^2$ ，大误差惩罚强； $RMSE=\sqrt{MSE}$ 与原量纲相同； $MAE=(1/m)\sum|\hat{y}-y|$ 对异常值更稳健； $R^2=1-SSE/SST$ ，越接近 1 越好，测试集上可为负。

6.8 欠拟合、过拟合与解决

情形	典型表现	原因	处理
欠拟合	训练误差、验证误差均高	模型太简单、特征不足、训练不充分、正则过强	增加特征/容量、训练更久、减弱正则、换更合适模型
过拟合	训练误差低，验证/测试误差高且差距大	模型过复杂、样本少/噪声大、训练过久、泄漏	更多数据/增强、K 折、早停、L1/L2、剪枝、Dropout、简化模型、降维

- **Early Stopping**：监控验证损失，连续若干轮无改善则停止并恢复最佳权重。
- 正则系数过大可能欠拟合；过小约束不足。K 折用于稳健估计/选参，本身不直接改变最终模型容量。

7. 线性回归、正则化、逻辑回归与极大似然

7.1 线性回归

- 模型： $\hat{y}=w^T x+b$ ；多元情况下决策面是超平面。目标为连续值预测。
- 常用目标： $J(w,b)=(1/2m)\sum(\hat{y}_i-y_i)^2$ ；1/2 便于求导。
- 正规方程（将偏置并入 X）： $\theta=(X^T X)^{-1} X^T y$ ，要求可逆；高维/大数据或奇异时用伪逆、正则化或梯度法。
- 梯度： $w \leftarrow w - \alpha(1/m)\sum(\hat{y}_i - y_i)x_i$ ， $b \leftarrow b - \alpha(1/m)\sum(\hat{y}_i - y_i)$ 。
- 多项式回归通过构造 $x^2, x^3, \dots$ 获得非线性拟合；阶数增加提升训练拟合能力，也会提高过拟合风险。

7.2 L1/L2 正则化

方法	目标中的惩罚	影响
Ridge/岭回归	$\lambda \sum w_j^2$ (L2)	平滑压小权重，通常不精确变零，处理共线性、减方差
Lasso/套索	$\lambda \sum$	w_j
Elastic Net	L1+L2	兼顾稀疏与稳定，适合相关特征成组存在

通常不惩罚偏置项。 λ 越大约束越强；过大导致欠拟合。

7.3 逻辑回归二分类

- 线性得分 $z = w^T x + b$ ；概率 $p = P(y=1|x) = \sigma(z) = 1/(1+e^{-z})$ ；阈值 0.5 时决策边界 $w^T x + b = 0$ 。
- 二元交叉熵/负对数似然： $J = -(1/m) \sum [y_i \ln p_i + (1-y_i) \ln(1-p_i)]$ 。
- 输出是类别概率；名称含“回归”源于对数几率建模： $\ln[p/(1-p)] = w^T x + b$ 。
- 线性逻辑回归产生线性边界；加入多项式/交互特征可形成非线性边界。

7.4 极大似然估计 MLE【原理题】

- 思想：“模型形式已定、参数未知”，选择最可能生成已观察样本的参数。
- 对独立样本，似然 $L(\theta) = \prod_i P(y_i|x_i;\theta)$ ；最大化乘积。取对数将乘积变求和： $\ell(\theta) = \sum \ln P(y_i|x_i;\theta)$ ；最大化对数似然等价于最小化负对数似然，二分类时正好得到交叉熵。
- 加 L2 后目标为交叉熵 + $\lambda ||w||^2$ ；正则过强会使权重过小并欠拟合。

7.5 多分类

- One-vs-Rest**：K 类训练 K 个二分类器，第 k 个判“第 k 类 vs 其余”，预测取概率/得分最大者。
- Softmax 回归**：对每类得分 z_k ， $p_k = e^{z_k} / \sum_j e^{z_j}$ ，所有概率和为 1；多分类交叉熵 $L = -\sum_k y_k \ln p_k$ 。

对比	线性回归	逻辑回归
任务	连续值回归	分类概率/标签
输出	任意实数	Sigmoid (0,1)；Softmax 概率分布
典型损失	MSE	交叉熵/负对数似然
决策边界	无分类边界	原始特征下通常线性

8. KNN 与支持向量机 SVM

8.1 K 近邻 KNN【模拟卷补充】

- 懒惰学习：训练阶段主要存数据；预测时计算待测点与训练样本距离，取最近 K 个，多数表决分类/均值回归。
- 欧氏距离： $d(x, z) = \sqrt{\sum_j (x_j - z_j)^2}$ ；也可用曼哈顿距离。
- 操作**：①预处理并标准化；②算所有距离；③升序取前 K；④分类多数投票（可按 $1/d$ 加权），回归取均值；⑤平票按距离和/先验/较小 K 规则处理。
- K 小：边界复杂、噪声敏感、方差高；K 大：更平滑、可能欠拟合；常用交叉验证选 K，二分类可取奇数避免平票。
- 缺点：预测慢、存储大、高维距离退化、尺度与无关特征敏感。

8.2 SVM 核心

- 监督分类模型，寻找能正确分开两类且**几何间隔最大**的超平面 $w^T x + b = 0$ ；预测 $\text{sign}(w^T x + b)$ 。
- 支持向量**：距离超平面最近、决定最优边界的训练样本；非支持向量通常不影响最终解，因此解具有稀疏性。
- 硬间隔： $\min 1/2 ||w||^2$ ，约束 $y_i(w^T x_i + b) \geq 1$ ；两条间隔边界间宽度为 $2/||w||$ 。
- Hinge 损失： $\max(0, 1 - y f(x))$ ；间隔外正确样本损失 0。

8.3 软间隔与 C

$\min 1/2 ||w||^2 + C \sum \xi_i$ ，约束 $y_i(w^T x_i + b) \geq 1 - \xi_i$ ， $\xi_i \geq 0$ 。

- 允许少量样本进入间隔或被错分，适合噪声、重叠与非完全线性可分数据，可提高泛化并减轻硬拟合。

- c 大：错误惩罚强，训练错误少、边界可能复杂/间隔窄，过拟合风险增；c 小：允许更多违例、间隔更宽、更平滑，过小会欠拟合。

8.4 核技巧、对偶与 Mercer 条件

- 线性不可分时，可将样本隐式映射到高维特征空间；核函数直接计算 $K(x,z)=\phi(x)^T\phi(z)$ ，无需显式展开高维坐标。
- 拉格朗日对偶把解写成训练样本内积的组合： $f(x)=\text{sign}[\sum_i \alpha_i y_i K(x_i,x)+b]$ ；KKT 条件使多数 $\alpha_i=0$ ， $\alpha_i>0$ 的样本构成支持向量。实际常用 SMO 分解求解；考试按教师提示无需现场推完整对偶证明。
- 合法 Mercer 核的直观条件：核对称，任意样本形成的 Gram 矩阵为半正定。

核	公式/特点
线性核	$K(x,z)=x^Tz$ ，高维稀疏、近线性数据
多项式核	$(\gamma x^Tz+r)^d$ ，描述特征交互
RBF/高斯核	$\exp(-\gamma$
Sigmoid 核	$\tanh(\gamma x^Tz+r)$ ，参数并非总满足 Mercer 条件

训练前通常标准化；通过交叉验证联合调 c,γ 。多分类可用 OVR 或 OVO。

8.5 SVM 与逻辑回归

维度	SVM	逻辑回归
学习目标	最大间隔，Hinge 损失	最大似然，交叉熵
输出	主要是分类得分，概率需校准	直接给条件概率
决定样本	主要由支持向量决定	所有样本均参与似然优化
非线性	核函数方便	需特征扩展或换模型
场景	小样本、高维、间隔明显	需概率解释、数据较大、线性可分趋势

两者均可含正则、均可形成线性边界；“SVM 永远优于 LR”错误。

9. K-means 聚类、K-means++、指标与 PCA

9.1 聚类基础

- 聚类属于无监督学习：希望**簇内相似度高、簇间相似度低**。簇可由中心、大小、密度、业务描述表示。主流方法还包括层次聚类（凝聚/分裂、树状图）、GMM（高斯混合概率簇）、DBSCAN（密度连通、可识别噪声且无需预设簇数）与谱聚类。
- 内部指标只用样本与簇结构；外部指标需真实标签/参考划分（如 ARI、NMI）。聚类编号本身无类别语义，若用 Accuracy/F1 与真实标签比较，先用匈牙利算法等做标签最佳匹配。外部成对指标可把样本对分成“两个划分均同簇、仅预测同簇、仅参考同簇、均不同簇”四类后计算。

9.2 K-means 【四步循环】

目标： $SSE = \sum_{k=1}^K \sum_{x_i \in C_k} ||x_i - \mu_k||^2$ 最小。

①随机选 K 个初始中心；②对每个样本，按欧氏距离分配给最近中心；③对每簇求所有样本均值，更新中心；④重复 ②—③，直到中心/分配不变、SSE 改善很小或达迭代上限。

- 每轮“分配+更新”不会增加 SSE，有限数据上收敛到局部最优；不同初始值可得到不同结果。
- 中心是均值，未必是某个真实样本；必须预先给 K；对特征尺度和异常值敏感。

9.3 K-means++ 与 K 的选择

- **K-means++**: 第一个中心随机选; 后续每个样本被选为新中心的概率与其到最近既有中心的距离平方 $D(x)^2$ 成正比, 使初始中心分散, 改善随机初值导致的差解; 仍不能保证全局最优。
- 选 K: 领域先验; 肘部法 (SSE 随 K 下降曲线拐点); 轮廓系数最大; Gap 统计量; 多指标结合。
- 轮廓系数: 对样本 i , $a(i)$ 为同簇平均距离, $b(i)$ 为与最近其他簇的平均距离; $s(i)=[b(i)-a(i)]/\max[a(i),b(i)] \in [-1,1]$, 越近 1 越好, 接近 0 位于边界, 负值可能错分。

9.4 适用与局限

适合: 连续数值、欧氏距离有意义、近球状/凸形、方差与大小相近、簇分离较清楚

不适合/需谨慎: 月牙/环形等非凸簇、密度/大小差异大、类别变量、异常值多、高维距离失效、K 未知

改进: 标准化、先处理异常值、多次随机初始化取最小 SSE、K-means++; 任意形状与噪声可考虑 DBSCAN, 概率椭圆簇可考虑 GMM, 层次结构可用层次聚类。

9.5 PCA 主成分分析【模拟卷补充】

- 无监督线性降维: 寻找方差最大的正交方向; 主成分是原特征的线性组合, 不一定对应某个原始特征; 降维通常有信息损失。
- **步骤**: ①中心化, 量纲差异大时先标准化; ②计算协方差矩阵; ③求特征值/特征向量; ④按特征值降序排列; ⑤选前 k 个 (累计解释方差常取 85%–95%); ⑥组成投影矩阵 W_k , 得 $Z=XW_k$ 。
- 优点: 去冗余、压缩、可视化、减轻共线性; 局限: 线性、解释性下降、最大方差未必最有利于分类、对异常值敏感。

10. 决策树、信息增益、剪枝与随机森林

10.1 基本结构与三个阶段

- 节点是用来划分走向的特征测试, 不同测试结果导向不同分支, 叶节点是类别/预测值; 预测时从根按条件走到叶。
- 构建采用贪心递归: 在当前节点选最优划分 $\rightarrow$ 分裂子集 $\rightarrow$ 对子集继续; 停止后赋叶标签。
- 三阶段: **特征选择、决策树生成、决策树剪枝**。

10.2 熵、条件熵与信息增益【核心计算】

- 熵: $H(X) = -\sum_{i=1}^n p_i \log_2 p_i$; 越大越混乱, 越小越纯; 其中 p_i 是第 i 类样本在集合中出现的概率, 注意 $p_i < 1$ 导致 $\log$ 得到负值。负号是为了把对数计算出来的负数变成正数; 二分类 $1/2-1/2$ 时最大为 1; 纯集为 0; 约定 $0\log 0=0$ 。
- 条件熵: 条件熵 $H(Y|X)$ 表示在已知特征 X 的条件下, 因变量 Y 的不确定性。
 $H(Y|X) = \sum_{x \in X} p(x)H(Y|X=x)$, $p(x)$ 即满足条件的比例, 整个公式即分隔后熵的加权和。
- 按特征 A 的第 v 个取值形成 D_v : $H(D|A) = \sum_v |D_v|/|D| \cdot H(D_v)$ 。
- 信息增益: $\text{Gain}(D,A)=H(D)-H(D|A)$; 越大表示分裂后纯度提升越多。 $H(D)$: 切分前总数据集 D 的熵;
 $H(D|A)$: 使用特征 A 切分后的条件熵; **信息增益越大, 说明用特征 A 来分类带来的“纯度提升”越高**。决策树在构建时, 每一群体分裂都会寻找信息增益最大的特征, “寻找”只是遍历 (ID3 只会这个), 或者使用二分查找 (C4.5+)。

计算模板: ①统计父集合各类比例并算 $H(D)$; ②按 A 分组; ③每组分别算熵; ④按组大小加权求条件熵; ⑤相减; ⑥保留 3 位小数并说明选择增益最大的特征。

模拟卷数值: D 中 4 正 2 负, $H(D)=0.918$; A 分成 3 个全正与 1 正 2 负, $H(D|A)=0.459$; $\text{Gain}=0.459$ 。

10.3 ID3、C4.5、CART

算法	划分依据	特点
ID3	最大信息增益	分类、多叉；偏好取值很多的特征，原版偏离散
C4.5	最大信息增益率 Gain/IV	多叉；缓解多值偏好，可处理连续、缺失； $\#IV(A) = -\sum_v (D_v / D) \log_2(D_v / D)$
CART	分类最小 Gini（基尼系数），回归最小平方误差	二叉树（组合离散特征）； $\text{Gini}(D) = 1 - \sum p_k^2$ ，越小越纯；基尼系数在 CART 中取代了熵计算，选择它是因为计算快；

10.4 过拟合、剪枝与随机森林

- 树易把噪声、孤立样本分得过细而过拟合；数据微小变化也可能产生完全不同结构。
- **预剪枝**：生长中按最大深度、最小样本数、最小纯度提升/验证收益提前停止；快但可能过早终止。
- **后剪枝**：先长较完整树，再依据验证集或复杂度代价删去收益小的子树；通常更稳，但训练开销大。
- **随机森林**：①Tree bagging：对训练集做多次有放回的随机取样，训练多棵树；②Feature bagging：节点分裂决策时仅从随机特征子集中选分裂，目的是进一步造成不同树的区别；①②分类多数投票、回归平均。两种随机性降低树间相关性与方差，通常比单树更抗过拟合。
- Bagging 可并行、主要降方差（在统计学中，如果有 N 个独立同分布的随机变量，它们的均值的方差是单变量方差的 $\frac{1}{N}$ ）；
- Boosting 串行强调前轮错误样本/残差，常降偏差但对噪声更敏感；AdaBoost 调高错分样本权重（第一轮做完，如果样本 A 被分类错误，第二轮时样本 A 的权重就会被调得极高），GBDT 只学残差（离目标的距离），XGBoost/LightGBM 是高效实现。

优点：可解释、处理非线性与特征交互、无需缩放、可混合数值/类别特征

缺点：易过拟合与不稳定、贪心局部最优、轴平行分割可能低效

11. 强化学习、MDP 与 Q-learning

11.1 要素与目标

- 智能体 Agent 在状态 s 选择动作 a ，环境转移到 s' 并返回奖励 r ；目标是学习策略 $\pi(a|s)$ 最大化长期累计回报。
- MDP 五元组 (S, A, P, R, γ) ：状态、动作、转移概率、奖励、折扣因子；马尔可夫性指给定当前状态后，未来与更早历史条件独立。
- 回报： $G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$ ； $\gamma=0$ 只看即时奖励， $\gamma \rightarrow 1$ 更重长期（无限时域通常取 <1 保证收敛）。
- $V^\pi(s)$ ：从状态 s 按策略 π 的期望回报； $Q^\pi(s, a)$ ：先做 a 后按 π 的期望回报。

11.2 Q-learning 【公式与步骤】

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

- 方括号内为 TD 误差； α 学习率； γ 未来奖励权重；目标使用下一状态的最大 Q ，因此 Q-learning 是**无模型、离策略 off-policy**方法。
- 步骤：①初始化 Q 表；②观察状态 s ；③用 ϵ -greedy 选 a ；④执行并获得 r, s' ；⑤按公式更新；⑥令 $s \leftarrow s'$ ，直至终止；⑦多回合训练。

11.3 探索与利用

- **探索**：尝试当前看来不最优的动作以发现新路径；**利用**：选择当前估计价值最高的动作。
- ϵ -greedy：概率 ϵ 随机动作，概率 $1-\epsilon$ 取 $\arg\max_a Q(s, a)$ ；常前期 ϵ 大、后期逐步衰减。
- 总走同一条次优路径：可能 ϵ 太小/衰减过快、 Q 初值悲观、奖励设计不当、学习率/折扣不合适；可增大或慢降 ϵ 、乐观初始化/UCB、奖励塑形、增加训练回合。

11.4 迷宫大题答法

- State：当前坐标，也可含障碍、方向、已访问信息；Action：上/下/左/右；Reward：到出口大正值（如 +100），撞墙负值（如 -10），每步小负值（如 -1）鼓励短路径，重复访问可加惩罚。
- 奖励塑形应避免让智能体通过循环刷分；终点设终止状态。
- Q 表无法推广到未见过/高维状态；DQN 用神经网络近似 Q。SARSA 使用实际下一动作，属 on-policy；策略梯度直接优化策略，适合连续动作。强化学习还可按“有模型/无模型”“基于价值/基于策略/Actor-Critic”“on-policy/off-policy”“Monte Carlo/时序差分 TD”分类。

12. 人工神经元、激活函数、学习规则与网络结构

12.1 神经元模型

人工神经元先做加权求和再过激活： $z=\sum_j w_jx_j+b=w^Tx+b$ ， $y=f(z)$ 。权重表示输入重要性，偏置移动阈值/决策面，非线性激活使多层网络能表达复杂函数；多层线性函数叠加仍等价于一层线性变换。

12.2 常见激活函数【曲线/梯度】

函数	公式/范围	梯度特点与用途
线性	$f(x)=x$ ，实数	导数 1；回归输出层；隐层全线性无意义
阶跃	$x\geq 0$ 输出 1 否则 0	几乎处处导数 0、阈值不可导，难用于梯度训练
Sigmoid	$1/(1+e^{-x})$ ，(0,1)	$f'=f(1-f)$ ；概率直观，饱和区梯度小、非零中心；二分类输出
tanh	$(e^x-e^{-x})/(e^x+e^{-x})$ ，(-1,1)	$f'=1-f^2$ ；零中心，但仍饱和/梯度消失
ReLU	$\max(0,x)$	正区导数 1，训练快、缓解消失；负区 0 可能“死亡”
Leaky/PreLU	负区保留小斜率	缓解死亡 ReLU；PreLU 负斜率可学习
ELU/GELU/SwiGLU	平滑或门控非线性	深层网络/Transformer 常见，选项识别即可
Softmax	$e^{z_k}/\sum e^{z_j}$	多分类输出为和 1 的概率分布

梯度消失常见于深层网络重复乘小权重/小导数，Sigmoid/tanh 饱和尤甚；梯度爆炸来自重复乘大于 1 的雅可比/权重。

12.3 三种基本学习规则

规则	典型更新	含义
Hebb	$\Delta w_{ij}=\eta y_i y_j$	同时激活则连接增强，无监督相关性学习
Delta/误差修正	$\Delta w_{ij}=\eta e_i x_j$	按输出误差沿负梯度调整，BP 的基础思想
Kohonen 竞争学习	获胜神经元 $w \leftarrow w+\eta(x-w)$	神经元竞争，胜者及邻域向输入靠近，用于自组织映射

12.4 网络结构

- 前馈网络：信息由输入到输出无环，如感知机、MLP/BP、CNN；适合静态映射。
- 反馈/循环网络：存在状态回路，如 RNN、Hopfield；能表达记忆、动态和序列。
- 网络能力由激活函数、连接拓扑、学习规则共同影响。典型网络还包括 RBF 网络、Hopfield 联想记忆网络、SOM 自组织映射、Boltzmann/受限 Boltzmann 机等。连续 Hopfield 用于 n 城市 TSP 的经典编码可设置 n^2 个神经元。深度学习通过多层学习分层特征，实现端到端表示学习；更深带来表达力，也增加优化、过拟合与算力难度。

13. BP 神经网络：参数、连接、计算量与梯度问题

13.1 MLP 与 BP

- 单层感知机只能处理线性可分问题；含非线性隐层的多层感知机可表达 XOR 等非线性关系。
- 第 l 层： $z^{(l)}=W^{(l)}a^{(l-1)}+b^{(l)}$ ， $a^{(l)}=f^{(l)}(z^{(l)})$ 。
- **BP**：前向计算输出与损失，反向利用链式法则逐层计算梯度，再用梯度下降更新参数；网络本身是前馈结构，误差在训练时反向传播。
- 通用逼近定理：具有足够隐层单元和合适非线性激活的单隐层网络可逼近一定条件下的连续函数；该定理不保证易训练、需要多少神经元或泛化好。

13.2 全连接层计数【核心】

对 $n_{in} \rightarrow n_{out}$ ：

- 含偏置可训练参数： $(n_{in} + 1)n_{out}$ ；无偏置： $n_{in}n_{out}$ 。
- 若课程把偏置视为连接，则连接数同参数数；若只算神经元间权连接，则为 $n_{in}n_{out}$ ，答题应说明口径。
- 乘加计算量课程常按 $FLOPs \approx 2 * n_{in}n_{out}$ （一次乘法+一次加法），通常忽略偏置与激活；全网求和。
- 网络总参数： $\sum_l (n_{l-1} + 1)n_l$ 。

例：14×14 灰度图→300→100→26：参数 $(14*14+1)*300+(300+1)*100+$

$(100+1)*26=59100+30100+2626=91826$ ；近似 $FLOPs$ $2(196*300+300*100+100*26)=182800$ 。

13.3 输出层/损失选择

任务	输出层	损失
二分类	1 个 Sigmoid	二元交叉熵
单标签 K 分类	K 个 Softmax	多分类交叉熵，标签常 One-Hot
多标签分类	K 个独立 Sigmoid	每标签二元交叉熵
回归	线性输出（或按范围限制）	MSE/MAE

如果二分类用 Sigmoid 配 MSE，在 BP 求导时，梯度中会包含 Sigmoid 的导数 $\sigma'(z)$ 。当预测值接近 0 或 1 时， $\sigma'(z)$ 会趋近于 0，导致**梯度消失**，模型停止学习。而如果使用 **Sigmoid + BCE**，在求导时中间项会神奇地**抵消掉**，最后的梯度仅与误差 $(y - \hat{y})$ 成正比。误差越大，梯度越大，学习越快。

13.4 BP 训练步骤

①初始化权重（不能全零/全相同，否则神经元对称）；②取样本/mini-batch；③前向传播；④计算损失；⑤由输出层向前按链式法则反传；⑥优化器更新；⑦在验证集监控并早停；⑧测试时关闭 Dropout、使用推理模式。

初始化：Xavier 常配 tanh/Sigmoid；He 常配 ReLU。**优化器**：SGD、Momentum、Adagrad、RMSProp、Adam 等；Adam 易用，SGD+动量有时泛化更好。**学习率**：可用指数/阶梯/余弦等衰减。

13.5 梯度消失/爆炸

- 原因：跨层链式乘积；权重谱半径和激活导数长期 <1 导致消失（梯度消失：使用了 Sigmoid 或 Tanh 等饱和激活函数；层数 n 很大；），长期 >1 导致爆炸（梯度爆炸：网络初始化时权重 w 设置过大；多个大于 1 的权重矩阵连续相乘）；网络深、Sigmoid/tanh 饱和、初始化不当、序列过长会加剧。
- 后果：前层几乎不更新、收敛慢；或参数/损失变为巨大值、NaN、训练震荡。
- 解决：ReLU/门控激活；Xavier/He 初始化；BatchNorm/LayerNorm；残差/跳连；合理学习率；梯度裁剪（爆炸）；LSTM/GRU（长序列）；缩短反传长度；数据标准化。

13.6 泛化与正则

- 过拟合：数据少、噪声、参数多、训练过久；方法：数据增强、L1/L2、Dropout、早停、简化网络、交叉验证。
- Dropout 训练时随机屏蔽神经元，推理时关闭并使用完整网络。
- 课件“浅层网络训练样本数最好多于参数数”可作为课堂经验启发，不是普适数学定理。

14. 卷积神经网络 CNN：尺寸、参数、连接与经典网络

14.1 核心概念

- 全连接图像网络参数巨大且忽略局部结构；CNN 使用**局部连接、权值共享、多卷积核**提取空间模式。
- 一个卷积核的深度必须等于输入通道 C_{in} ；有 C_{out} 个卷积核就输出 C_{out} 张特征图。
- **感受野**：输出单元在原图上可受影响的区域；叠加小卷积、步长、池化、空洞卷积均会扩大有效感受野。
- 池化层：通过减小特征图的大小来减少计算复杂性。它通过选择池化窗口内的最大值或平均值来实现。这有助于提取最重要的特征。
- 偏置 (Bias) 加在**卷积核与输入局部窗口做完乘加运算 (点积) 之后，进入激活函数 (如 ReLU) 之前**。例： $Output_1 = A + B + C + b_1$ (A、B、C 是三个卷积核通道输出，对应 RGB)。
- 输出通道只取决于核数量，因为输入通道在卷积已合并。如果多层堆叠，只有第一层通道数取决于输入通道数，后续所有通道数取决于前一层卷积核个数。

14.2 输出尺寸【必算】

一般卷积/池化 (k 核 s 步长 d 空洞卷积膨胀间隔 c 通道数)：

$$H_{out} = \text{floor}[(H_{in} + 2p_h - d_h(k_h - 1) - 1) / s_h] + 1$$

$$W_{out} = \text{floor}[(W_{in} + 2p_w - d_w(k_w - 1) - 1) / s_w] + 1$$

普通膨胀率 $d=1$ 时： $\text{floor}[(H+2p-k)/s]+1$ 。输出通道：卷积为卷积核数量 C_{out} ；池化通常保持通道数。

- $\text{stride} > 1$ 或无填充卷积会缩小；池化核大于步长/不填充也缩小。
- 奇数核、步长 1 要保持尺寸常取 $p=(k-1)/2$ (same padding)。
- Padding 保留边缘信息；Dilation 在参数不增的情况下扩大感受野。

14.3 参数、神经元、连接与 FLOPs【区分口径】

对卷积核 $k_h \times k_w$ ，输入 C_{in} ，输出 C_{out} ：

- 卷积层的参数量 (含每核一个偏置)： $(k_h k_w C_{in} + 1) C_{out}$ ；无偏置删 +1。
- 输出神经元： $H_{out} W_{out} C_{out}$ 。
- 连接数：每个输出神经元连接 $k_h k_w C_{in}$ 个输入 (若计偏置再 +1)，故 $Connections = H_{out} \times W_{out} \times C_{out} \times (k_h \times k_w \times C_{in} + 1)$ 。
- FLOPs 课程近似： $FLOPs \approx 2 \times H_{out} \times W_{out} \times C_{out} \times (k_h \times k_w \times C_{in})$ ，通常忽略偏置/激活。
- **参数量因权值共享与空间尺寸无关；连接数和计算量随输出空间尺寸增长。**

池化：输出尺寸套同一公式；标准 Max/Avg Pool **无可训练参数**，输出神经元 $H_{out} W_{out} C$ ；比较大小/求和也有计算，但通常不按卷积参数算法。早期 LeNet 的“可训练下采样”可含缩放与偏置，题干若明确才计参数。

全连接：输入展开长度 $N = HWC$ ；含偏置参数 $(N+1)N_{out}$ ，FLOPs 近 $2NN_{out}$ 。

14.4 标准计算模板

①写输入 $H \times W \times C_{in}$ ；②套公式算 H_{out}, W_{out} ；③输出图数 = C_{out} ；④参数按卷积核体积 $\times$ 核数 + 偏置；⑤神经元 = 输出体积；⑥连接 = 每神经元局部输入数 $\times$ 输出神经元；⑦FLOPs = 乘加 $2 \times$ 局部乘法数；⑧池化后更新 H/W、不改 C；⑨Flatten 后算 FC。

模拟卷： $64 \times 64 \times 3$ ，10 个 5×5 ， $s=1, p=2$ ，含偏置：输出 $64 \times 64 \times 10$ ；参数 $(5 \times 5 \times 3 + 1) \times 10 = 760$ ； $2 \times 2, s=2$ 池化后 $32 \times 32 \times 10$ ，Flatten = 10240；8 分类 FC 参数 $(10240 + 1) \times 8 = 81928$ ；Softmax。

例： $14 \times 14 \times 3$ ，5 个 5×5 ， $s=1, p=1$ ，无偏置：输出 $(14 + 2 - 5) + 1 = 12$ ，即 $12 \times 12 \times 5$ ；参数 $5 \times 5 \times 3 \times 5 = 375$ ；连接 $75 \times 12 \times 12 \times 5 = 54000$ ； $2 \times 2, s=2$ 池化后 $6 \times 6 \times 5$ ，神经元 180；4 类 FC 无偏置参数 $180 \times 4 = 720$ 。

14.5 为什么 CNN 优于纯全连接图像网络

- 局部连接和权值共享大幅减少参数、存储和过拟合风险；利用图像局部空间相关性；逐层学习边缘→纹理→部件→语义；池化/卷积提供一定平移鲁棒性；可处理不同层次视觉任务。

14.6 经典网络与“用深度换宽度”

网络	核心特点
LeNet	早期手写数字网络：卷积—下采样—全连接
AlexNet	ReLU、Dropout、数据增强、重叠最大池化、GPU 推动深度视觉
VGG	连续 3×3 小卷积堆叠，结构规则；2 个 3×3 感受野≈5×5，3 个≈7×7
GoogLeNet/Inception	多尺度并行 1×1/3×3/5×5/池化；1×1 先降通道减少计算；全局平均池化、辅助分类器
ResNet	$y=F(x)+x$ 残差/恒等捷径，改善深网退化和梯度传播
Xception/MobileNet	深度可分离（depthwise separable）卷积：逐通道空间卷积 + 1×1 通道混合，显著降参数/FLOPs

VGG 参数比较（输入输出通道均 C）：1 个 7×7 参数 $49C^2$ ；3 个 3×3 参数 $27C^2$ ，感受野同为 7×7 且增加 3 次非线性。1 个 5×5 为 $25C^2$ ；2 个 3×3 为 $18C^2$ 。若各层通道不同，应逐层按 $k^2C_{in}C_{out}$ 算。

Inception 降维例：输入 128 通道，直接 256 个 3×3 参数 $128 \times 256 \times 9 = 294912$ ；先 1×1 将 128→32，再 3×3 将 32→256，总参数 $128 \times 32 + 32 \times 256 \times 9 = 77824$ 。

ResNet 机制：恒等支路让梯度可直接传播，并让残差分支只学习 $F(x)=H(x)-x$ ；当新增层无用时可趋近 0，深度增加不必导致训练误差反而上升。瓶颈块常为 1×1 降维→3×3→1×1 升维。

14.7 提升网络性能的因素

数据规模/质量与增强、网络深度和宽度、感受野与多尺度、特征工程/归一化、初始化、优化器和学习率、正则化/Dropout、归一化层、残差连接、算力与训练时间。深/宽增加并非无条件更好，需兼顾优化和泛化。DenseNet 强调特征复用，SqueezeNet/MobileNet 强调轻量化，NAS 自动搜索结构；预训练+迁移学习可在小数据上微调。视觉任务包括分类、目标检测、语义/实例分割、关键点、跟踪等。

15. 生成对抗网络 GAN

15.1 结构与目标

- **生成器 G：**把噪声 $z \sim p_z$ 映射为伪样本 $G(z)$ ；**判别器 D：**输出样本来自真实数据的概率。
- 二者对抗：D 尽量区分真/假，G 尽量骗过 D；理想平衡时生成分布接近真实分布，D 难以区分。
- 判别模型学习 $P(y|x)$ 或分类边界；生成模型学习数据分布并能采样。原始目标： $\min_G \max_D E_{\{x \sim p_{data}\}} [\log D(x)] + E_{\{z \sim p_z\}} [\log(1 - D(G(z)))]$ 。当真实与生成分布几乎不重叠时，JS 散度型目标可能给 G 很弱梯度；WGAN 用 Wasserstein 距离改善连续性。
- 训练交替进行：固定 G 更新 D（真实标签 1、生成标签 0）→固定 D 更新 G（推动 $D(G(z))$ 变大）→重复。训练时损失耦合；推理时 G 可独立生成，D 也可单独用于判别/特征任务。

15.2 特点、应用与 DCGAN

- 生成模型学习数据分布，可用于图像生成、修复、超分辨率、风格/域转换、文本到图像、视频、数据增强。
- DCGAN 常以步幅卷积替代池化、减少全连接、使用 BatchNorm；G 隐层 ReLU/输出 tanh，D 常用 LeakyReLU。

15.3 缺点与模式崩溃【重点】

- **模式崩溃：**G 只生成少数几种样本，却能反复骗过 D，覆盖不了真实分布的多样模式。原因包括对抗目标不稳定、G/D 能力或学习速度失衡、梯度信息差、G 找到少量“投机”输出。

- 其他问题：训练振荡/不收敛、梯度消失、超参数敏感、评价困难、生成伪影。
- 缓解：平衡 G/D 更新次数和学习率；改善架构/归一化；增加多样性约束或多生成器；使用 WGAN/WGAN-GP 等更平滑距离与梯度惩罚；充分数据、谱归一化、标签平滑、监控样本多样性。有限技巧不能保证完全消除。

16. 循环神经网络 RNN、LSTM 与 GRU

16.1 RNN 基础

- CNN 重点建模空间局部结构；RNN 重点建模序列顺序和上下文，适合文本、语音、时间序列、情感分类。
- 共享参数展开： $h_t = \phi(W_{xh} x_t + W_{hh} h_{t-1} + b_h)$ ； $y_t = g(W_{hy} h_t + b_y)$ 。当前状态由**当前输入 x_t 与上一记忆状态 h_{t-1}** 共同决定。
- 共享权重使模型可处理可变长序列；结构可为一对多、多对一、同步多对多、异步多对多（编码器—解码器）。
- BPTT 将网络沿时间展开后反向传播；重复乘循环雅可比导致长程梯度消失/爆炸，普通 RNN 难记长期依赖。

16.2 词表示

- One-Hot 高维稀疏、无语义距离；Embedding 用可训练矩阵把词索引映射为稠密向量，输入 $[B, S]$ 可变为 $[B, S, D]$ 。
- Word2Vec：CBOW 用上下文预测中心词，Skip-gram 用中心词预测上下文；GloVe 利用全局共现。静态词向量对同一词只给一个表示，难处理一词多义。

16.3 LSTM 【理解门控】

设拼接 $[h_{t-1}, x_t]$ ：

- 遗忘门 $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$ ：保留多少旧记忆；
- 输入门 $i_t = \sigma(W_i[\dots] + b_i)$ ，候选 $\hat{c}_t = \tanh(W_c[\dots] + b_c)$ ；
- 记忆更新 $c_t = f_t \odot c_{t-1} + i_t \odot \hat{c}_t$ ；
- 输出门 $o_t = \sigma(W_o[\dots] + b_o)$ ；隐藏 $h_t = o_t \odot \tanh(c_t)$ 。

细胞状态提供较直接的信息通路，缓解长期梯度问题；LSTM 仍可能梯度异常且参数较多，单元参数约为同尺寸普通 RNN 的 4 组门/候选矩阵。

16.4 GRU 简化

- 合并细胞状态 c_t 与隐藏状态 h_t ；取消独立输出门；用**更新门 z** 和**重置门 r** 控制记忆，因此参数约为 LSTM 的 3/4、训练更快。
- 一种课程常用写法： $r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$ ； $\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h)$ ； $z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$ ； $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$ 。不同教材对 z 的两项系数可能互换，答题按课件口径说明即可。
- 堆叠 RNN 增加层次；双向 RNN 同时利用前后文，但在线自回归生成时无法直接看到未来。

17. 注意力机制与 Transformer

17.1 注意力通式与打分函数

给定查询 q 、键 k_i 、值 v_i ：先算分数 $e_i = \text{score}(k_i, q)$ ，再 $a_i = \text{softmax}(e_i)$ ，最后上下文 $c = \sum_i a_i v_i$ 。

打分	形式
加性/Bahdanau	$v^T \tanh(Wk + Uq)$
点积	$k^T q$
缩放点积	$k^T q / \sqrt{d_k}$ ，防止维度大时点积过大导致 Softmax 饱和

打分	形式
双线性/General	$k^T W q$

软注意力对所有位置加权且可微；硬注意力选择离散位置，通常需采样/强化学习估计。

17.2 自注意力 Q/K/V

- 同一序列线性投影得到 $Q=XW_Q, K=XW_K, V=XW_V$ ；Q 表示“我要找什么”，K 表示“我能与什么匹配”，V 是被汇总的信息。
- $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$ 。
- 自注意力可直接建模任意位置依赖并行计算；缺点是没有天然顺序信息，标准注意力时间/显存约 $O(n^2)$ ，长序列成本高，局部先验较弱。
- 多头： $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$ ，将各头拼接再投影；不同头可学习不同关系/子空间。

17.3 位置编码 PE

Transformer 无循环/卷积顺序，需注入位置信息。正弦位置编码：

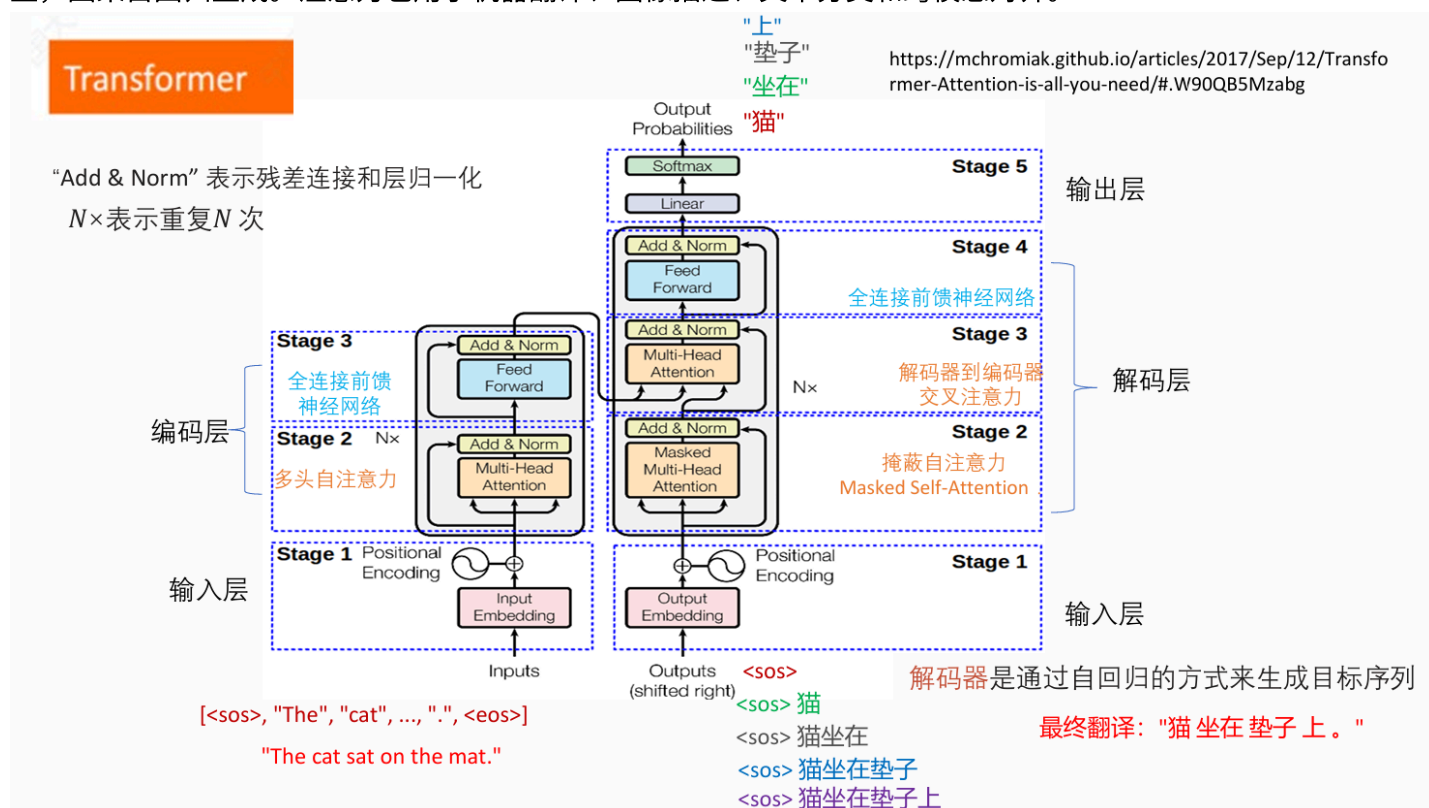
$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{(2i/d_{\text{model}})})$$

$$\text{PE}(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{(2i/d_{\text{model}})})$$

也可用可学习位置向量或相对/旋转位置编码。位置编码与词向量相加，使模型区分顺序和相对距离。

17.4 Transformer 结构

- Encoder 层**：多头自注意力 → Add & LayerNorm → 逐位置 FFN → Add & LayerNorm。
- Decoder 层**：Masked 多头自注意力（禁止看未来）→ 编码器—解码器交叉注意力 → FFN，各子层配残差与 LN；自回归逐 token 生成。
- FFN**：对每个位置独立应用同一两层非线性网络，如 $\text{FFN}(x) = W_2 \sigma(W_1 x + b_1) + b_2$ ，负责通道维变换与非线性表达。
- Residual/Add**：保留原信息、提供短梯度路径；**LayerNorm**：对单样本/单 token 的特征维归一化，适合变长序列与小批量。
- Mask**：Padding mask 屏蔽填充；causal mask 屏蔽未来 token。
- BERT 以编码器为主，双向表征，课件示例采用 WordPiece 分词及 [CLS]、[SEP] 特殊标记；GPT 以解码器为主，因果自回归生成。注意力也用于机器翻译、图像描述、文本分类和跨模态对齐。



18. 大语言模型 LLM：概念、训练、局限与优化

18.1 概念与结构

- LLM 是在大规模文本/代码等语料上训练、参数规模较大、以 Transformer 为主干的语言模型；核心预训练目标通常是根据上下文预测 token。
- 输入文本先经 Tokenizer 切分为 token，再查 Embedding、叠加位置信息，经多层 Transformer，最后输出词表上的概率分布。
- 能力包括上下文学习、指令跟随、生成、总结、问答、一定推理与工具调用；能力会随模型、数据、算力共同扩展。Scaling law 强调三者需匹配，盲目只增参数效率不一定高。
- 架构可为 Dense 或 MoE：MoE 由路由器为每个 token 激活少数专家，以稀疏计算扩大总容量；路由负载均衡和通信是难点。课件中的扩展概念：MLA 通过低秩/压缩表示降低注意力 KV 缓存开销，MTP 让模型同时预测多个未来 token 作为训练/解码辅助。

18.2 训练流程【三大阶段/四个细步】

- **三大阶段口径**：①大规模自监督预训练；②监督指令微调 SFT；③对齐训练。
- **四细步口径**：①预训练；②SFT/Instruction Tuning；③训练奖励模型/偏好模型；④RLHF 或其他偏好优化，使输出更符合人类偏好和安全要求。
- 预训练学通用统计与知识；SFT 学任务格式和指令响应；对齐阶段用人类/AI 偏好数据优化有用性、诚实性和安全性。指令模型面向通用指令遵循；推理模型强化多步思考与问题求解，两者能力有重叠。

18.3 局限

局限	表现/来源
Token 化与统计生成	模型预测下一 token，可能掌握表面相关性但缺少稳定的符号验证
知识时效	预训练知识有截止时间，训练后事件未知或过时
上下文/记忆有限	超出上下文的信息会丢失；长期会话记忆依赖外部系统
幻觉	生成流畅但错误、虚构来源/事实/数字；置信语气不代表正确
推理与计算不可靠	多步逻辑、精确算术、约束执行可能出错
偏见、安全、隐私	继承数据偏见，可能泄露/生成有害内容，受提示注入影响
成本与可解释性	训练推理耗算力，内部决策机制难完全解释

18.4 幻觉原因与避免【潜在大题】

- 原因：训练资料缺失/冲突/过时；噪声和偏见；下一 token 目标偏向“语言合理”；问题含糊或提供上下文不足；采样温度过高；模型无外部事实锚定；长上下文信息遗失；错误在多步生成中累积。
- 缓解：明确任务、范围、格式与“不确定时说明”；提供可靠上下文；要求来源/逐项核验；降低温度；使用搜索、数据库、计算器和代码工具；RAG；领域微调与偏好对齐；事实校验/二次模型/规则检查；关键领域保持人工复核。任何单一手段都不能绝对消除幻觉。

18.5 三类通用优化

方法	做法	适合/代价
Prompt Engineering	明确角色、目标、上下文、约束、示例、输出格式，分解任务	无需改权重、成本低；受上下文与模型能力限制
Fine-tuning	用领域/任务数据更新权重；可全量或 LoRA/Adapter 参数高效微调	固化行为/风格；需高质量数据与训练，知识更新不够灵活

方法	做法	适合/代价
RAG	查询→向量/关键词检索→取相关文档→拼接上下文→生成并可附出处	更新快、可溯源、减少知识型幻觉；依赖切分、Embedding、召回、重排和文档质量

RAG 向量检索常用余弦相似度 $\cos(a,b)=a \cdot b / (||a|| ||b||)$ ；需防“检索不到、检索错、模型忽略证据”三类失败。

18.6 AI Agent

- LLM Agent 结合模型、工具、记忆与工作流：**Profile/角色目标、Memory、Planning、Action/工具调用**。开发流程可概括为角色/目标定义→接入知识库与工具→编排工作流→测试评估→发布与监控。
- 基本闭环：接收任务→观察环境/检索记忆→规划→调用工具或执行动作→获取结果→反思/更新→输出。
- 与纯聊天模型相比，Agent 能调用搜索、代码、数据库、日历等外部工具并持续执行；风险包括工具误用、提示注入、权限过大、错误累积，应采用最小权限、确认关键动作、沙箱、日志和停止条件。在线 API 部署方便但受隐私、成本与网络限制；本地部署可增强隐私、可控和离线能力，但需要硬件、量化/推理优化和运维。人机协作强调理解模型边界、整合领域知识、设计指令以及人工判断输出可靠性。

19. 模拟卷与课件边缘考点补充

19.1 贝叶斯定理与贝叶斯网络

- 贝叶斯公式： $P(H|E)=P(E|H)P(H)/P(E)$ ；先验 $P(H)$ ，似然 $P(E|H)$ ，证据 $P(E)$ ，后验 $P(H|E)$ 。
- 贝叶斯网络是有向无环图 DAG：节点为随机变量，边表示直接条件依赖；联合分布分解为 $P(X_1, \dots, X_n)=\prod_i P(X_i|Pa(X_i))$ ；利用条件独立降低参数量，可在给定证据后推后验。既可处理离散变量，也可扩展连续/混合变量。

19.2 关联规则

- 支持度： $support(A \rightarrow B)=P(A \cup B)=count(A \cup B)/N$ ，表示 A、B 同时出现频率。
- 置信度： $confidence(A \rightarrow B)=P(B|A)=support(A \cup B)/support(A)$ ，表示含 A 的事务中也含 B 的比例。
- 提升度： $lift=P(B|A)/P(B)$ ；>1 正关联，=1 近独立，<1 负关联。

19.3 模拟退火

- 单解随机搜索，模拟金属降温；若新解更好则接受，若更差 $\Delta E > 0$ ，以概率 $p=\exp(-\Delta E/T)$ 接受。
- 高温：更易接受差解、全局探索强；降温后趋向局部改进。冷却太快易困局部，太慢计算长；有限步不保证全局最优，也不属于种群算法。

19.4 TF-IDF

- $TF(t,d)$ 衡量词 t 在文档 d 的频率； $IDF(t)=\log[N/df(t)]$ （常做平滑）； $TF-IDF=TF \times IDF$ 。
- 全语料普遍词因 df 大而降权；在少数文档频繁出现的区分性词获高权。它不直接编码词序和深层语义。

19.5 不确定性与模糊推理（题目样例覆盖，低优先级）

- CF 可信度**：证据合取常取 min，析取取 max；规则 IF E THEN H (CF_rule) 对结论贡献常写 $CF(H)=CF_rule \times \max(0, CF(E))$ ；两个正证据合成可用 $CF_{12}=CF_1+CF_2-CF_1CF_2$ 。
- 模糊集合**：元素属于集合的程度由隶属函数 $\mu_A(x) \in [0,1]$ 给出；模糊表示概念边界，不等于逻辑混乱。
- 最大—最小合成/Mamdani**：规则关系常取 $\mu_R(x,y)=\min[\mu_A(x), \mu_B(y)]$ ；输入 A' 推理输出 $\mu_{B'}(y)=\max_x \min[\mu_{A'}(x), \mu_R(x,y)]$ ；最后可用最大隶属度、重心等解模糊。
- Dempster-Shafer**：基本概率分配 m(A) 分配给命题集合；Bel(A) 为对 A 的确定支持下界，Pl(A) 为可能支持上界，区间 [Bel, Pl] 表示不确定范围。

20. 高频计算模板、速判与答题句式

20.1 一页公式表

主题	公式
A*	$f(n)=g(n)+h(n)$
UCT	$w_i/N_i+c\sqrt{\ln N_p/N_i}$
PSO	$v \leftarrow wv+c1r1(pbest-x)+c2r2(gbest-x) ; \quad x \leftarrow x+v$
梯度下降	$\theta \leftarrow \theta - \alpha \nabla J$
Sigmoid	$1/(1+e^{-z})$
Softmax	$p_k=e^{\{z_k\}}/\Sigma e^{\{z_j\}}$
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$
Precision / Recall	$TP/(TP+FP) ; \quad TP/(TP+FN)$
F1	$2PR/(P+R)=2TP/(2TP+FP+FN)$
熵	$-\Sigma p_k \log_2 p_k$
条件熵/增益	$\Sigma_v (D_v / D)H(D_v); H(D)-H(D A)$
Gini	$1-\Sigma p_k^2$
K-means	$\Sigma_k \Sigma \{x \in C_k\} x-\mu_k ^2$
SVM 软间隔	$\min 1/2 w ^2+C\Sigma \xi_i$
Q-learning	$Q \leftarrow Q+\alpha[r+\gamma \max_{Q'} Q-Q]$
FC 参数	$(n_{in}+1)n_{out}$
Conv 输出	$\text{floor}[(H+2p-d(k-1)-1)/s]+1$
Conv 参数	$(k_hk_wC_{in}+1)C_{out}$ （含偏置）
Conv 连接	$(k_hk_wC_{in}+[1])H_{out}W_{out}C_{out}$
Conv FLOPs	$2k_hk_wC_{in}H_{out}W_{out}C_{out}$
Attention	$\text{softmax}(QK^T/\sqrt{d_k})V$
Bayes	$P(H E)=P(E H)P(H)/P(E)$

20.2 计算题统一检查清单

1. 先写公式与符号含义，再代数值；2. 确认“含不含偏置”；3. 卷积先算 H/W，再通道、参数、神经元、连接、FLOPs；4. 熵每个分支按样本数加权；5. 混淆矩阵先确定谁是正类；6. PMX 中段保持不动，映射链直到不冲突；7. α - β 写遍历顺序和触发时 α, β ；8. A* 同分按题目顺序并保留父节点；9. 最后标单位、类别与结论。

20.3 高频速判

说法	结论
DFS 一定完备/最优	错
BFS 在有限分支、有限目标深度下完备；等代价时最优	对
A* 的 h=0 退化为 UCS；等权时可与 BFS 一致	对
A* 任意启发函数都保证最优	错，需可采纳/一致等条件
α - β 会改变 Minimax 选择	错
MCTS 必须有精确人工评价函数	错
GA 变异是主要重组手段	错，交叉重组，变异维持多样性
PSO 只依赖群体最优	错，还有惯性和个体最优

说法	结论
测试集可用于反复调参	错
Accuracy 总适合不平衡数据	错
逻辑回归用于连续值回归	通常错，它是分类模型
SVM 解主要由支持向量决定	对
K-means 可自动发现任意形状簇和 K	错
决策树通常不需特征标准化	对
PCA 使用标签并保留原特征	错，无监督且主成分是线性组合
强化学习每步都有正确标签	错，主要由奖励反馈
多层线性激活可表达任意非线性	错
标准最大池化有可训练权重	错，题干特殊说明除外
卷积参数量随图像 H/W 增大	同核与通道下不变；连接/FLOPs 增大
Softmax 用于互斥多分类，Sigmoid 用于二分类/多标签	对
GRU 比 LSTM 多一个独立细胞状态	错，GRU 合并状态并减少门
Transformer 自带顺序信息	错，需位置编码
LLM 的流畅回答必然正确	错

20.4 简答题“定义—机制—优缺点—应用”模板

- **定义**：该方法解决何类问题、输入输出是什么。
- **机制**：用 2–4 个关键词或公式说明怎样工作。
- **作用/优点**：为何有效、改善什么。
- **局限/条件**：适用假设、参数风险、何时失效。
- **应用/例子**：结合题目场景落地。完全不会时也先写相关定义、公式、步骤和合理结论，避免空白。

Python 速查

题干关键词	正确结论
<code>list.append()</code> 后函数内又 <code>data = [...]</code>	<code>append</code> 改原列表；重新赋值只改局部变量绑定
查看 DataFrame 前 n 行	<code>df.head(n)</code>
查看 DataFrame 后 n 行	<code>df.tail(n)</code>
<code>plt.show()</code>	显示当前图形，不是清空图形
清空 Matplotlib 当前图形	<code>plt.clf()</code> ；清空坐标轴 <code>plt.cla()</code>
Scikit-learn 训练模型	<code>model.fit(X, y)</code>
Scikit-learn 预测	<code>model.predict(X)</code>
Scikit-learn 预处理转换	<code>transform</code> / <code>fit_transform</code>
NumPy 普通切片	多数情况返回 <code>view</code> ，共享原数组数据
NumPy 花式索引/布尔索引	多数情况返回 <code>copy</code>
<code>a * b</code> 对 NumPy 矩阵	元素乘法，不是矩阵乘法
NumPy 矩阵乘法	<code>a @ b</code> 或 <code>np.dot(a, b)</code>
<code>axis=0</code>	沿行方向压缩，得到每列结果

题干关键词	正确结论
axis=1	沿列方向压缩，得到每行结果
Pandas 删除列	df.drop(..., axis=1)
Pandas 删除行	df.drop(..., axis=0)
Pandas 标签索引	df.loc[row_label, col_label]
Pandas 位置索引	df.iloc[row_pos, col_pos]

1. Python 基础语法

```
# ===== 变量与基本规则 =====
x = 3                # 动态类型；无需声明 int x
x = "abc"            # 同一变量可重新绑定到其他类型
PI = 3.14            # 只是约定常量，Python 不强制 const
None                # 类似 null/nil；判断用 x is None

# ===== 运算符 =====
5 / 2                # 2.5；真除法，不是 C++ int/int
5 // 2               # 2；整除，向下取整
5 ** 2               # 25；幂运算
3 < x < 10           # 链式比较合法
and_or = (x and True) or False # 逻辑运算用 and/or/not，不用 && || !
# ===== == 与 is =====
a = [1, 2]; b = [1, 2]; c = a
print(a == b, a is b, a is c) # True False True
# ===== 真值判断 =====
bool(0), bool(""), bool([]), bool({}), bool(None) # 都是 False
bool(1), bool("0"), bool([0])                    # 都是 True
# ===== 输入输出/格式化 =====
name, age = "Li", 20
print(f"{name}: {age}") # Li: 20
```

2. 容器：字符串、列表、元组、字典、集合

```
# ===== 字符串 str: 不可变 =====
s = "hello"
s[0]                # 'h'
s[-1]               # 'o'
s[1:4]              # 'ell'；左闭右开
s[::-1]             # 'olleh'
s.upper()           # 'HELLO'
"a,b,c".split(",")  # ['a', 'b', 'c']
"-".join(["a", "b"]) # 'a-b'
# ===== 列表 list: 可变，类似动态数组 =====
lst = [1, 2, 3]
lst.append(4)        # [1, 2, 3, 4]；原地修改
lst.extend([5, 6])   # [1, 2, 3, 4, 5, 6]
lst.insert(0, 9)     # [9, 1, 2, 3, 4, 5, 6]
lst.pop()            # 删除并返回最后一个元素
lst[1:3] = [7, 8]    # 切片赋值，原地改
[x * x for x in range(4)] # [0, 1, 4, 9]
# ===== 元组 tuple: 不可变，常用于多返回值 =====
t = (1, 2, 3)
a, b = 1, 2          # 解包/多重赋值
return_value = (a, b) # 函数多返回值本质常是 tuple
# ===== 字典 dict: 键值对 =====
```

```

d = {"name": "Li", "age": 20}
d["age"]          # 20; 不存在会 KeyError
d.get("score", 0)  # 不存在返回默认值 0
d["score"] = 95    # 新增/修改
for k, v in d.items(): pass
# ===== 集合 set: 去重、集合运算 =====
s1, s2 = {1, 2, 3}, {3, 4}
s1 & s2            # {3} 交集
s1 | s2            # {1, 2, 3, 4} 并集
s1 - s2            # {1, 2} 差集

```

3. 流程控制、循环、函数

```

# ===== for / range / enumerate / zip =====
for i in range(3):          # 0, 1, 2; stop 不包含
    pass
for idx, val in enumerate(["a", "b"]):
    pass                    # idx=0/1, val='a'/'b'
for x, y in zip([1, 2], [3, 4]):
    pass                    # (1,3), (2,4)
range(n)                    # [0, n)
range(a, b)                 # [a, b)
range(a, b, c)              # 步长 c
# ===== 可变对象传参: 考试高频 =====
def modify(data):
    data.append(4)           # 修改原 list
    data = [5, 6]           # 仅让局部变量 data 重新绑定
x = [1, 2, 3]
modify(x)
print(x)                    # [1, 2, 3, 4]
# ===== 默认参数坑: 不要用可变对象作默认值 =====
def bad(x, buf=[]):
    buf.append(x)
    return buf
bad(1); bad(2)              # [1, 2]; buf 被多次调用共享
def good(x, buf=None):
    if buf is None:
        buf = []
    buf.append(x)
    return buf
# ===== 作用域 =====
x = 1
def f():
    global x                 # 修改全局变量才需要 global
    x = 2

```

4. 面向对象基础

```

class Animal:
    kind = "animal"          # 类属性: 所有对象共享
    def __init__(self, name): # 构造函数; self 类似 this, 必须显式写
        super(TargetClassName, ClassInstance).MethodName() # 多态
        self.name = name     # 实例属性
    def speak(self):
        return f"{self.name} speaks"
class Dog(Animal):           # 继承
    def speak(self):         # 方法重写

```



```

A * B          # 对应元素相乘，不是矩阵乘法
np.vstack((A, B))    # 竖向拼接
np.hstack((A, B))    # 横向拼接
np.split(np.arange(6), 2)          # 分成 2 段
np.savetxt("out.txt", A)
np.loadtxt("out.txt")

```

6. Pandas 速查

```

import pandas as pd
import numpy as np
# ===== Series / DataFrame =====
s = pd.Series([22, 26, 24], index=["张", "李", "段"], name="年龄")
s.values          # 底层值，通常是 NumPy 数组
s.index           # 行索引
s.dtype           # 元素类型
s.name            # Series 名称
df = pd.DataFrame({
    "年龄": [22, 26, 24],
    "城市": ["北京", "成都", "南京"],
    "通过": [True, False, True]
}, index=["张", "李", "段"])
df.index          # 行索引
df.columns        # 列索引
df.dtypes         # 每列类型
df.values         # 二维 ndarray
df.head(3)        # 前 3 行；考试高频
df.tail(1)        # 后 1 行
df.sample(2)      # 随机抽 2 行
# ===== 文件读写 =====
df.to_csv("data.csv", index=False)
df2 = pd.read_csv("data.csv")
pd.read_excel("data.xlsx")
pd.read_json("data.json")
# ===== 选行选列：推荐 loc / iloc =====
df["年龄"]        # 取一列，返回 Series
df[["年龄", "城市"]] # 取多列，返回 DataFrame
df[1:3]           # 行切片，位置 1 到 2
df.loc["张", "年龄"] # 标签索引：行标签、列标签
df.loc[["张", "李"], ["年龄", "城市"]]
df.iloc[0, 0]     # 位置索引：第 0 行第 0 列
df.iloc[:2, :1]   # 前 2 行、前 1 列
df[df["年龄"] > 23] # 布尔筛选
# ===== 删除/缺失值 =====
x = df.drop(["通过"], axis=1) # 删除列；axis=1
y = df.drop(["张"], axis=0)   # 删除行；axis=0
pd.isnull(df)                # 缺失值位置
df_clean = df.dropna(axis=0)  # 删除含缺失值的行
df_fill = df.fillna(df.mean(numeric_only=True)) # 数值列均值填充
# ===== 聚合/分组 =====
df.describe()               # 数值列统计摘要
df["年龄"].mean()
df.sum(axis=0, numeric_only=True) # 按列聚合
df.sum(axis=1, numeric_only=True) # 按行聚合
g = pd.DataFrame({"班级": ["A", "A", "B"], "分数": [80, 90, 70]})
g.groupby("班级")["分数"].mean() # 分组平均
g.groupby("班级").agg({"分数": [np.sum, np.mean, np.std]})
# ===== 合并 =====

```

```

left = pd.DataFrame({"id": [1, 2], "name": ["a", "b"]})
right = pd.DataFrame({"id": [1, 3], "score": [90, 80]})
pd.merge(left, right, on="id", how="inner") # 内连接; 默认 inner
pd.merge(left, right, on="id", how="left") # 左连接
pd.concat([left, left], axis=0) # 纵向拼接
pd.concat([left, right], axis=1) # 横向拼接

```

7. Matplotlib 速查

```

import matplotlib.pyplot as plt
import numpy as np
# ===== 折线图 =====
x = np.linspace(0, 2 * np.pi, 50)
plt.plot(x, np.sin(x), "r-^", label="sin") # 红色 实线 三角标记
plt.plot(x, np.cos(x), "g--", label="cos") # 绿色 虚线
plt.legend() # 显示图例
plt.xlabel("x")
plt.ylabel("y")
plt.title("Sin and Cos")
plt.grid()
plt.show() # 显示图形; 不是清空
plt.clf() # 清空当前 figure
plt.cla() # 清空当前 axes
plt.close() # 关闭图形窗口
# ===== 常见图形 =====
plt.scatter(x, np.sin(x)) # 散点图
plt.show()
plt.hist(np.random.randn(1000), bins=50) # 直方图
plt.show()
plt.pie([100, 250, 300], labels=["apple", "orange", "pear"])
plt.savefig("pie.png") # 保存图像; 通常放在 show 前
plt.show()
# ===== 子图 =====
plt.subplot(2, 1, 1) # 2 行 1 列, 第 1 个区域
plt.plot(x, np.sin(x), "r")
plt.subplot(2, 1, 2) # 第 2 个区域
plt.plot(x, np.cos(x), "g")
plt.show()

```

8. Scikit-learn 速查

```

# ===== 常用导入 =====
from sklearn.datasets import load_iris, load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import accuracy_score, r2_score, mean_squared_error

# ===== 统一 API: 考试高频 =====
# model.fit(X_train, y_train) # 训练模型
# model.predict(X_test) # 预测
# model.score(X_test, y_test) # 模型自带评分
# transformer.fit_transform(X_train) # 预处理: 训练并转换训练集
# transformer.transform(X_test) # 用训练集参数转换测试集

# ===== 典型监督学习流程 =====
data = load_iris()

```

```

X, y = data.data, data.target          # X 通常二维; y 通常一维
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)  # 只在训练集 fit
X_test = scaler.transform(X_test)        # 测试集只 transform
clf = LogisticRegression(max_iter=200)
clf.fit(X_train, y_train)                # 训练
pred = clf.predict(X_test)               # 预测
accuracy_score(y_test, pred)             # 分类准确率
clf.score(X_test, y_test)                # 通常也是准确率
# ===== 回归流程 =====
d = load_diabetes()
X_train, X_test, y_train, y_test = train_test_split(
    d.data, d.target, test_size=0.2, random_state=42
)
reg = LinearRegression()
reg.fit(X_train, y_train)
y_pred = reg.predict(X_test)
r2_score(y_test, y_pred)                 # 回归常用 R^2
mean_squared_error(y_test, y_pred)       # 均方误差

```

9. 串联模板

```

# ===== NumPy + Pandas + Matplotlib + sklearn 一体化模板 =====
import ...
# 1. 读数据
df = pd.read_csv("data.csv")
df.head()
df.info() if hasattr(df, "info") else None
# 2. 清洗/选特征
df = df.dropna(axis=0)
X = df.drop(["label"], axis=1).values
y = df.loc[:, "label"].values
# 3. 切分
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# 4. 标准化
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
# 5. 建模
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print(accuracy_score(y_test, pred))
# 6. 可视化
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.xlabel("feature 1")
plt.ylabel("feature 2")
plt.title("Data")
plt.show()

```